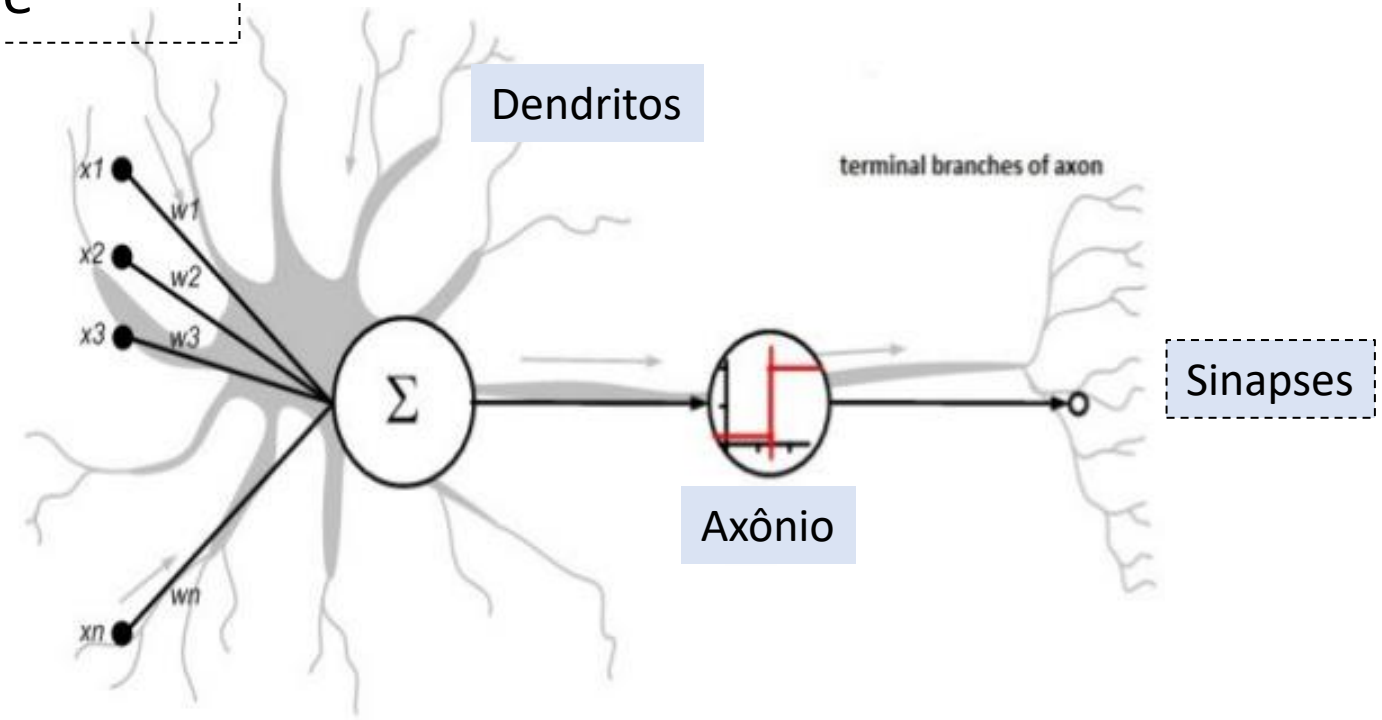
# REDES NEURAIS

Neurônios – MCP e Perceptron

Prof. Tacla UTFPR/Curitiba

# REDES NEURAIS

Uma coleção massivamente paralela de unidades de processamento pequenas e simples, onde as interligações formam a maior parte da "inteligência" da rede




unidades de processamento = neurônios
interligações com outros neurônios = sinapses

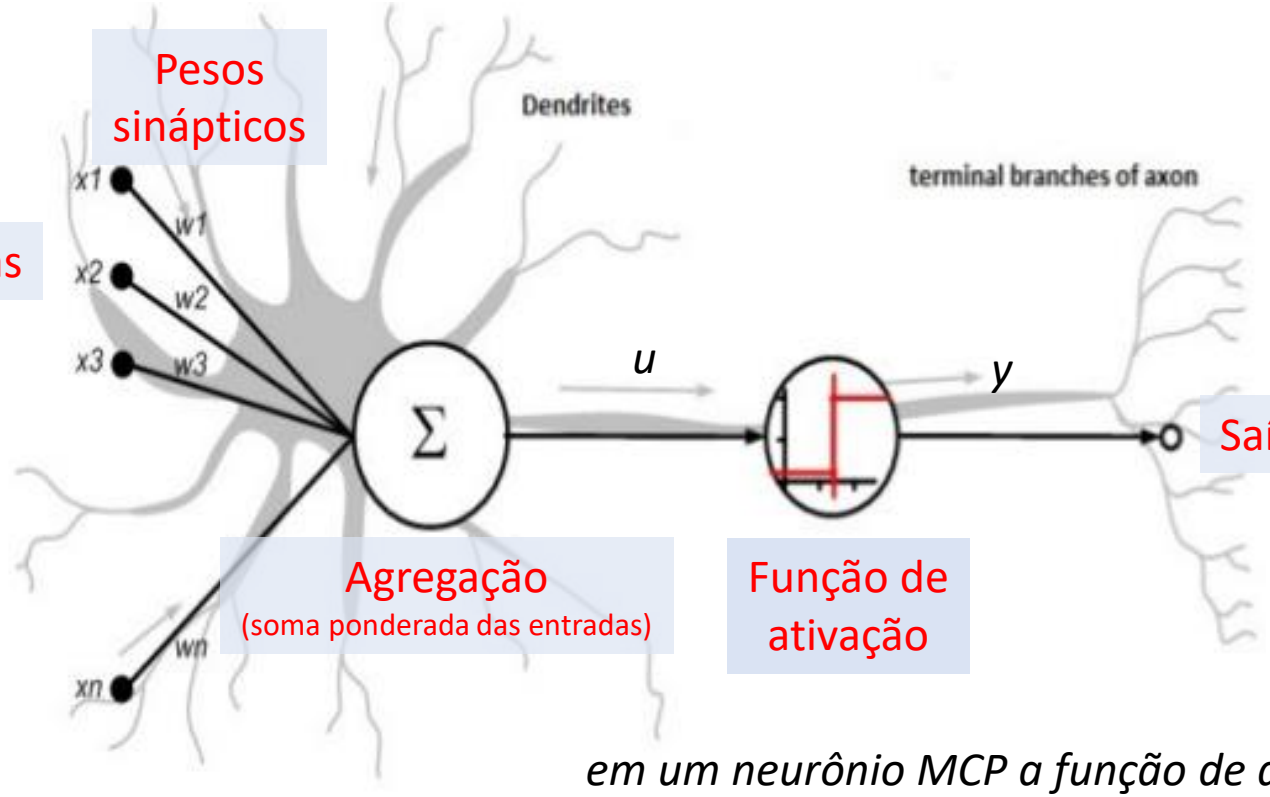
*Neurônio real e neurônio artificial*

# de McCulloch-Pitts (MCP)

Neurônio Artificial

# Neurônio Artificial

Os primeiros trabalhos na área datam de 1943: **McCulloch e Pitts** desenvolveram o primeiro modelo matemático inspirado nos neurônios biológicos.

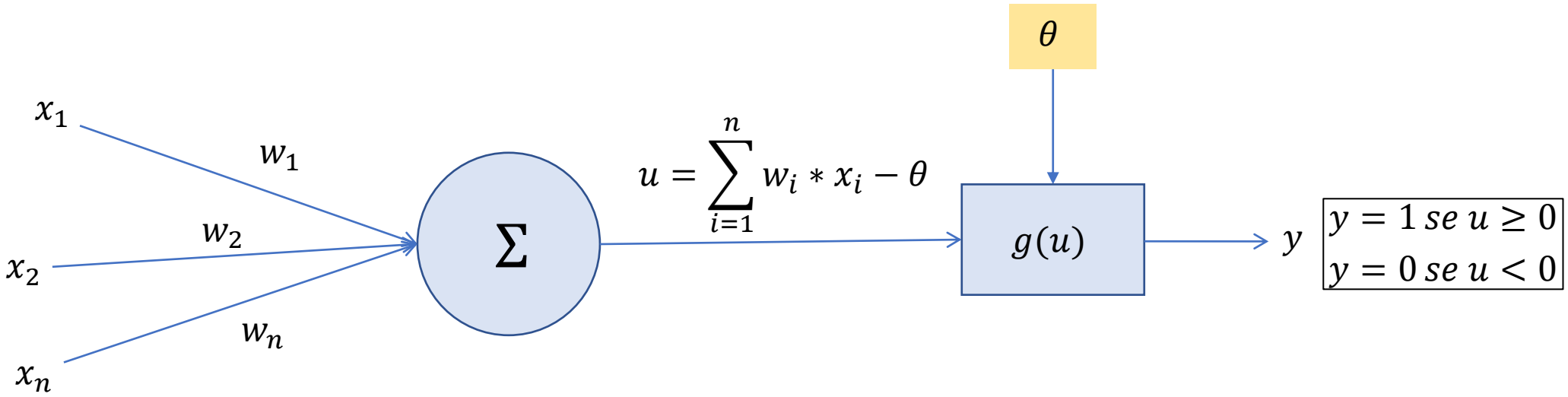



$y \in \{0,1\}$

$$y = 1 \; se \; \left(\Sigma(x_j * w_j)\right) \geq \boldsymbol{\theta}$$
$$y = 0 \; se \; \left(\Sigma(x_j * w_j)\right) < \boldsymbol{\theta}$$

*em um neurônio MCP a função de ativação é degrau*

# Neurônio Artificial de McCulloch-Pitts

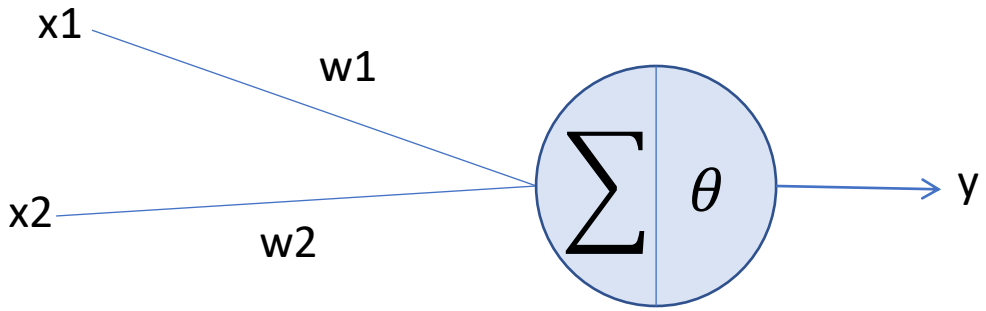


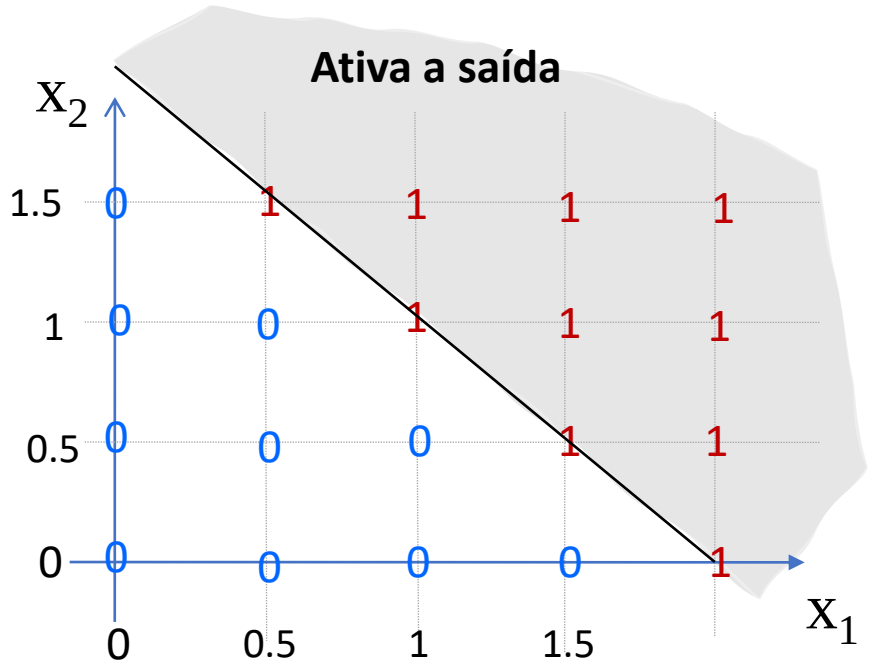

$$y = 1\, se\, \sum_{i=1}^{n} w_i * x_i \geq \theta$$
$$y = 0\, se\, \sum_{i=1}^{n} w_i * x_i < \theta$$

# Neurônio MCP (separação do espaço de entradas)

Considerando o caso particular de 1 neurônio com 2 entradas:

*o neurônio cria uma separação linear no espaço de entradas*




$$y = \begin{cases} 1 & \text{se } w_1 x_1 + w_2 x_2 \geq \theta \\ 0 & \text{caso contrário} \end{cases}$$

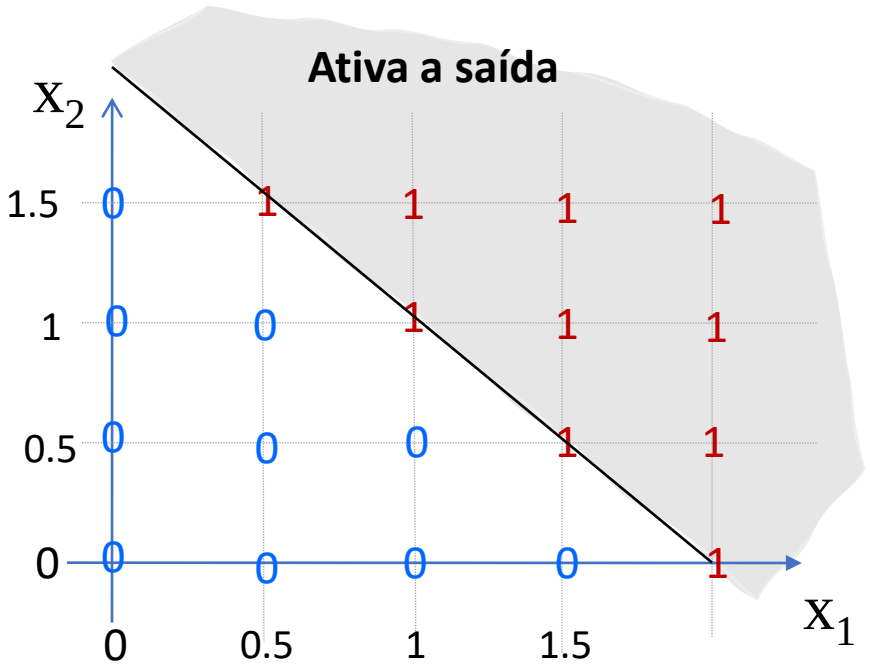
*Assumindo w1=w2=1 e Θ=2*

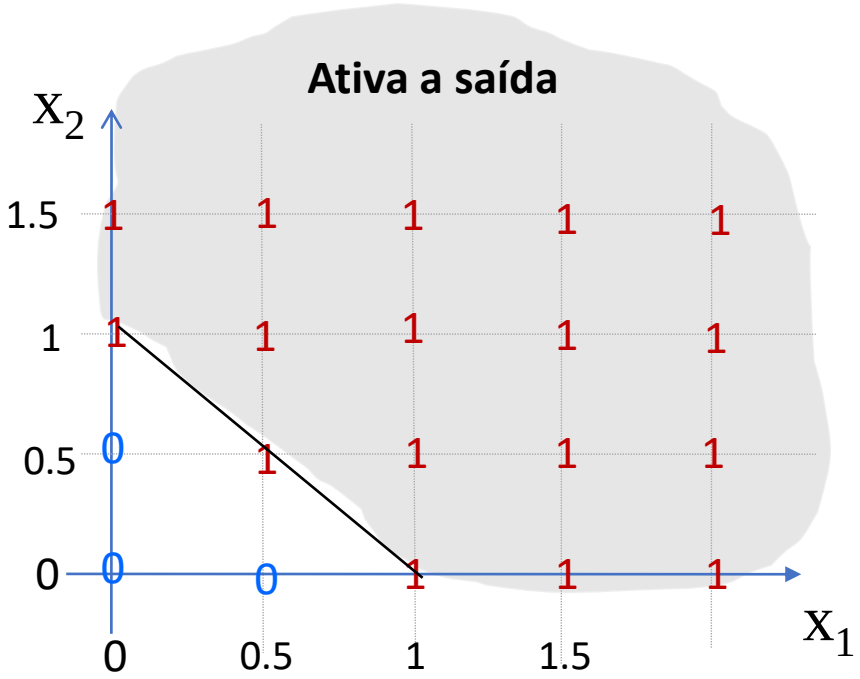
Inclinacao da reta: w1,w2

Translação da reta: Θ

$$y = \begin{cases} 1 & \text{se } 1*x_1 + 1*x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

# Neurônio MCP (separação linear do espaço de entradas)



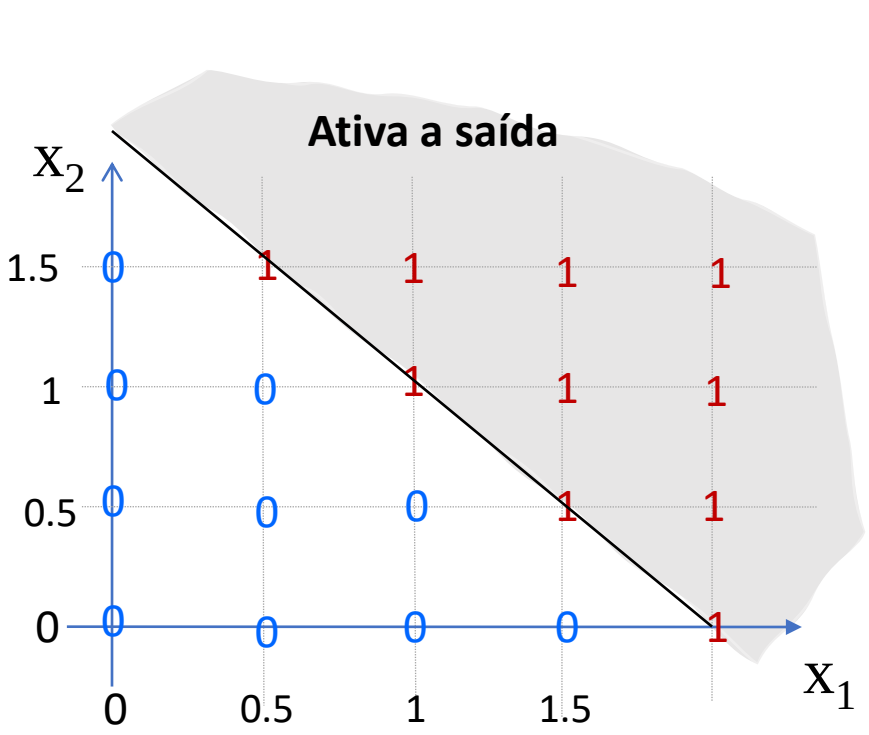

$$y = \begin{cases} 1 & \text{se } 1*x_1 + 1*x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

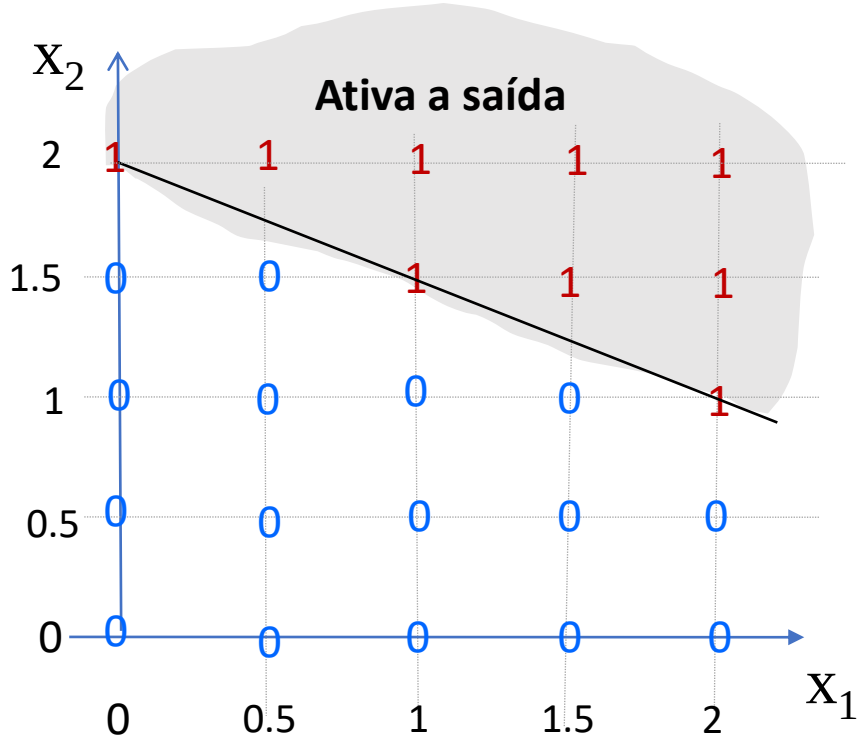
*Fazendo w1=w2=1 e* ***Θ=1***

Muda a translação da reta: Θ

$$y = \begin{cases} 1 & \text{se } 1*x_1 + 1*x_2 \geq 1 \\ 0 & \text{caso contrário} \end{cases}$$

# Neurônio MCP (separação linear do espaço de entradas)




$$y = \begin{cases} 1 & \text{se } 1*x_1 + 1*x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

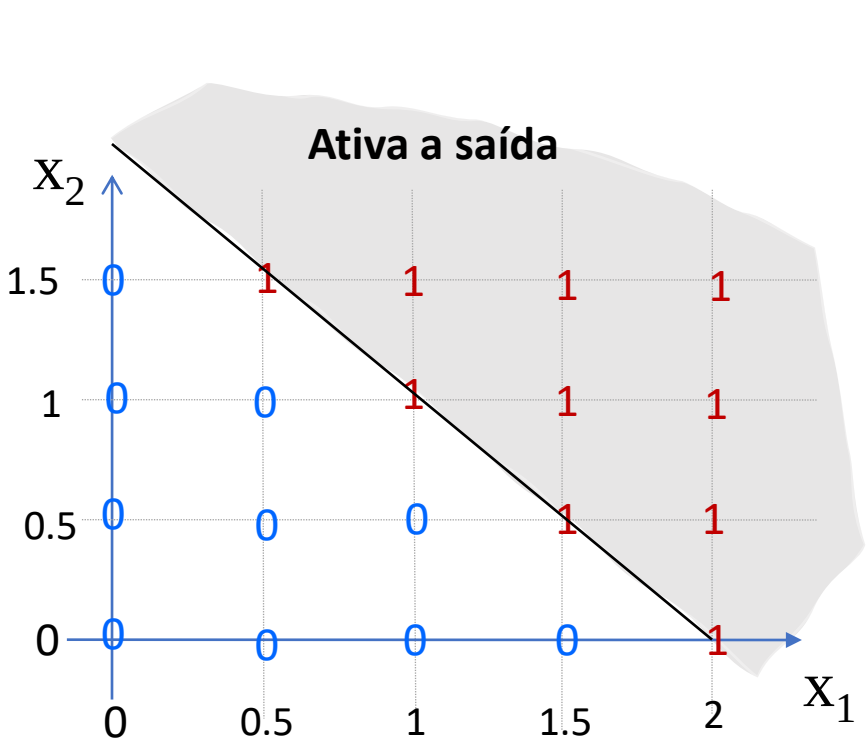
*Fazendo w1=0.5 w2=1 e* ***Θ=2***

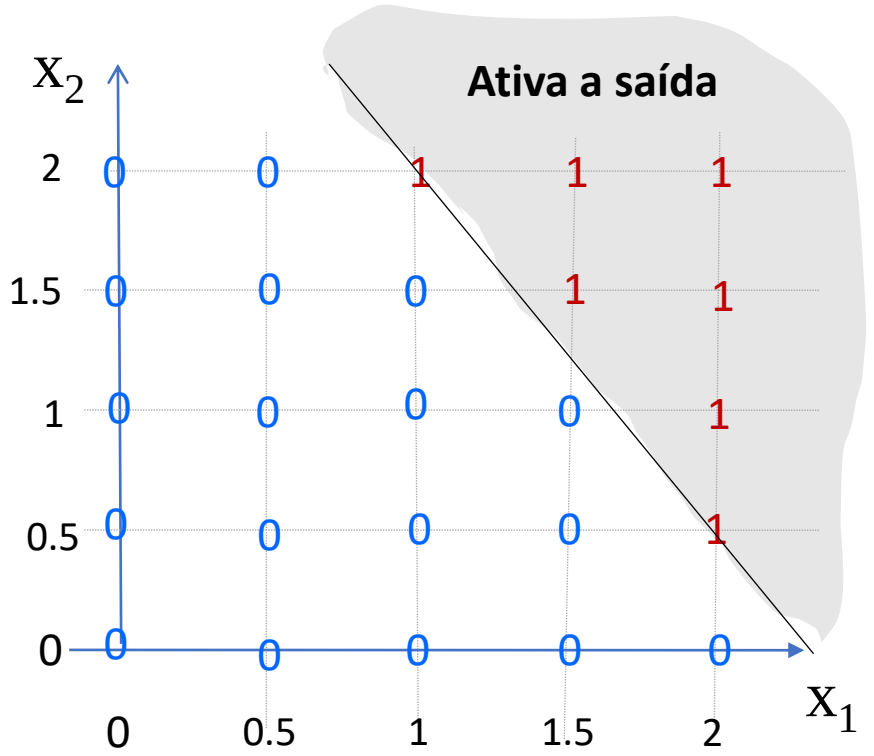
$$y = \begin{cases} 1 & \text{se } 0.5 * x_1 + 1 * x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

*Alterar w1, muda a inclinação*

*A entrada 1 tem metade da influência que a 2*

# Neurônio MCP (separação linear do espaço de entradas)




$$y = \begin{cases} 1 & \text{se } 1*x_1 + 1*x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

*Fazendo w1=1 w2=0.5 e* ***Θ=1***

$$y = \begin{cases} 1 & \text{se } 1*x_1 + 0.5*x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

*Alterar w2, muda a inclinação*

*A entrada 1 tem o dobro da influência que a 2*

# Neurônio MCP (separação do espaço)

define um plano inclinado que depende dos pesos...

*que é 'truncado' pela limiar Θ (Θ =2) da função degrau*

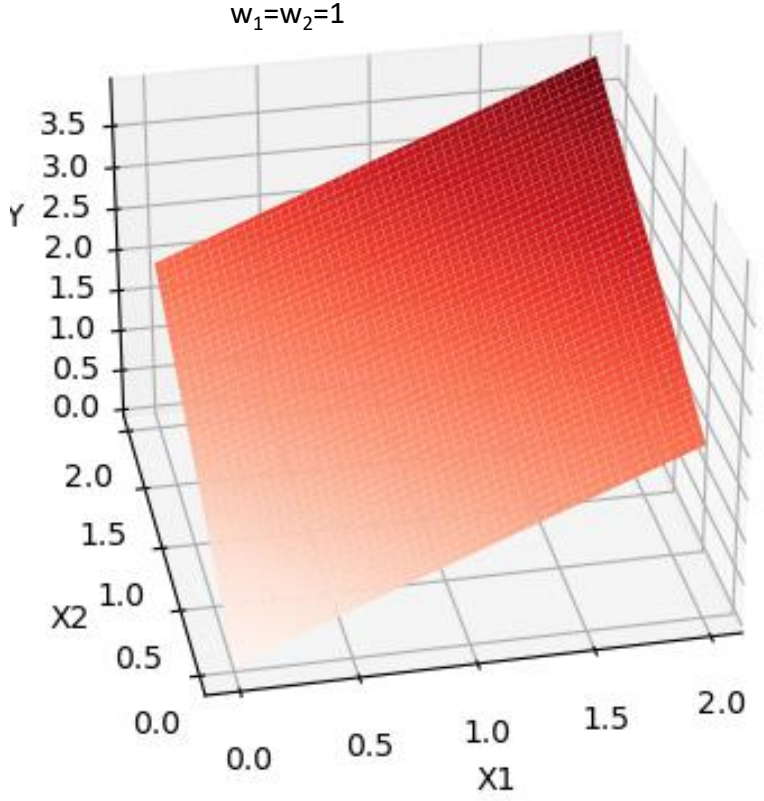
*para ativar (ou não) a saída*

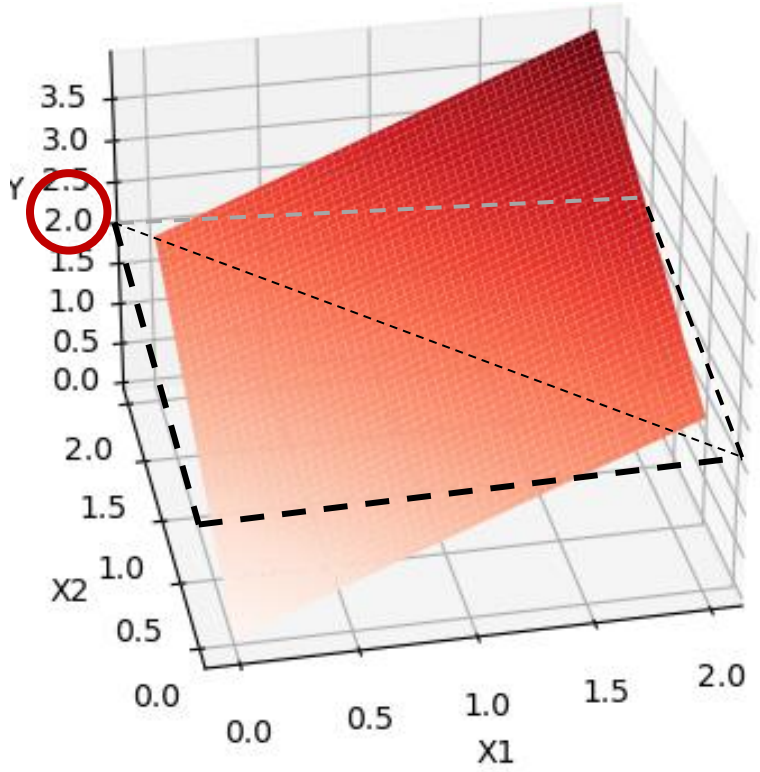
$$y = \begin{cases} 1 & \text{se } 1 * x_1 + 1 * x_2 \geq 2 \\ 0 & \text{caso contrário} \end{cases}$$

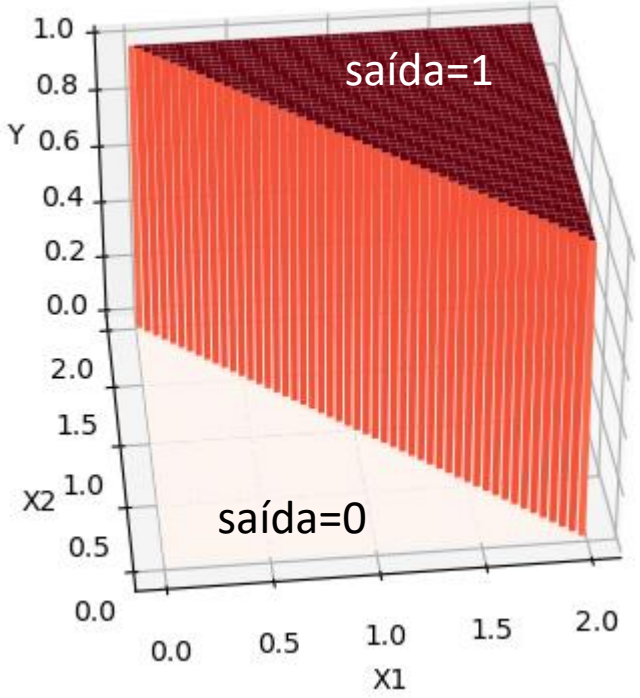
$w_1 x_1 + w_2 x_2$

$w_1 x_1 + w_2 x_2 \geq 2$

$w_1$=$w_2$=1



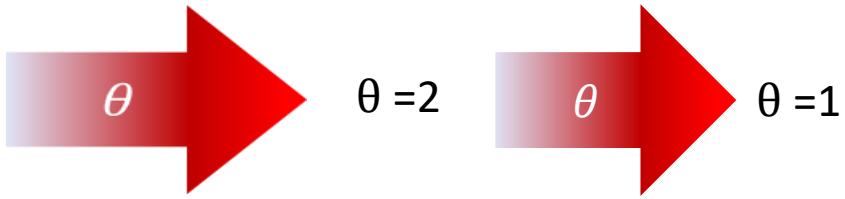



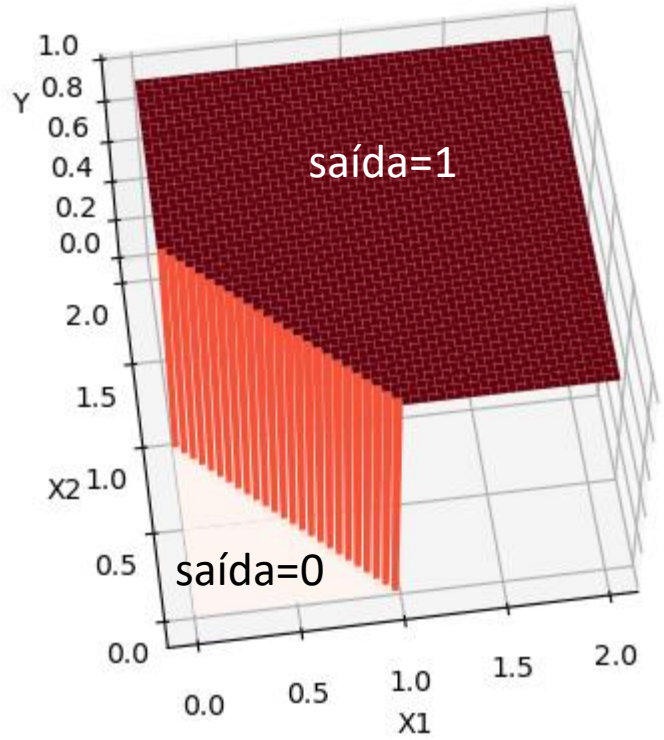

$\theta$ define um plano paralelo ao plano $x1 \times x2$

*hiperplano da saída*

# Neurônio MCP (separação do espaço)

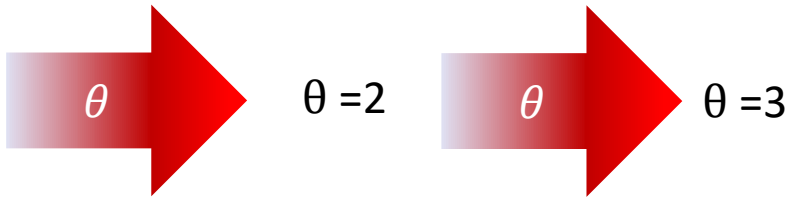
Variações no limiar transladam o separador

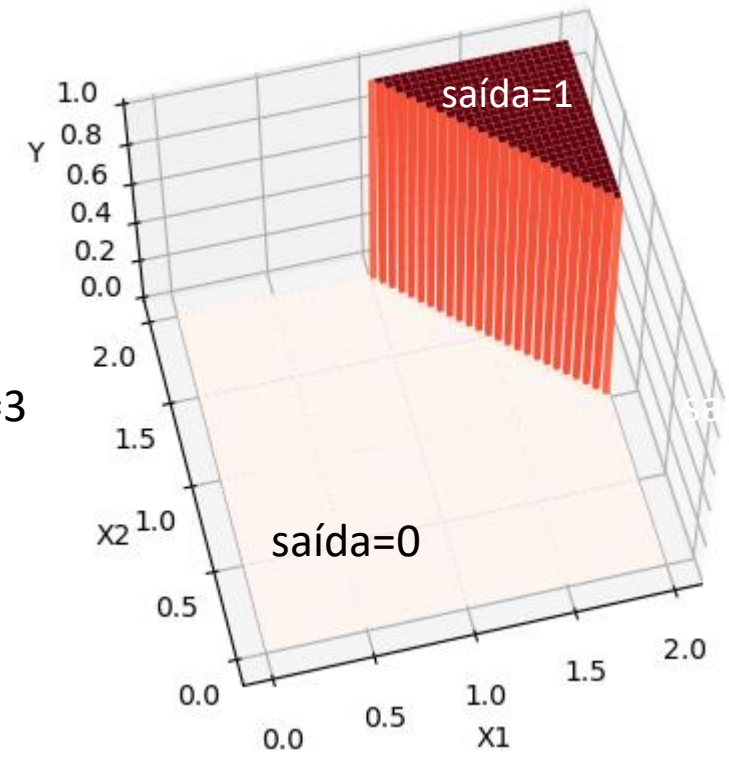


# Neurônio MCP (separação do espaço)

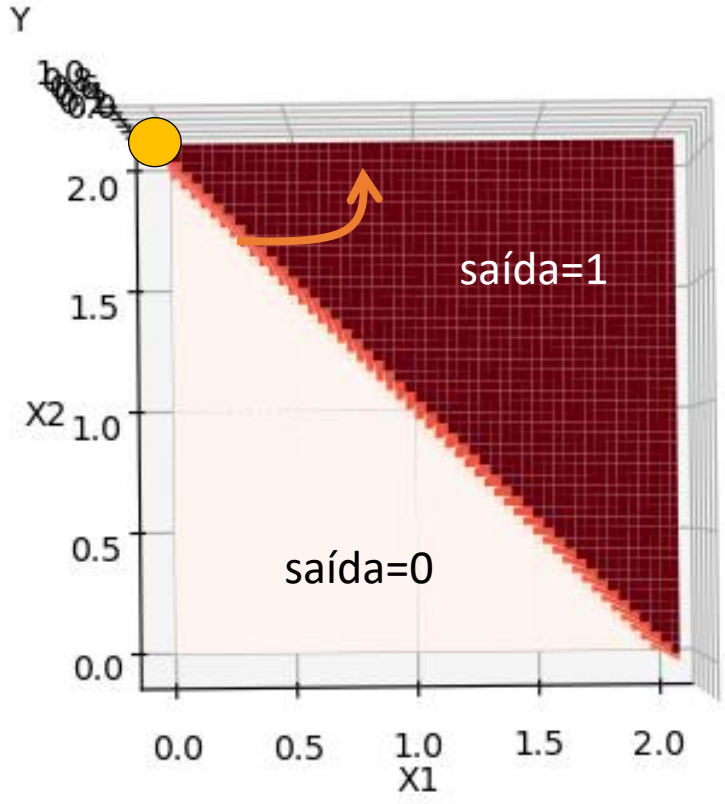
Variações no limiar transladam o separador



# Neurônio MCP (separação do espaço)

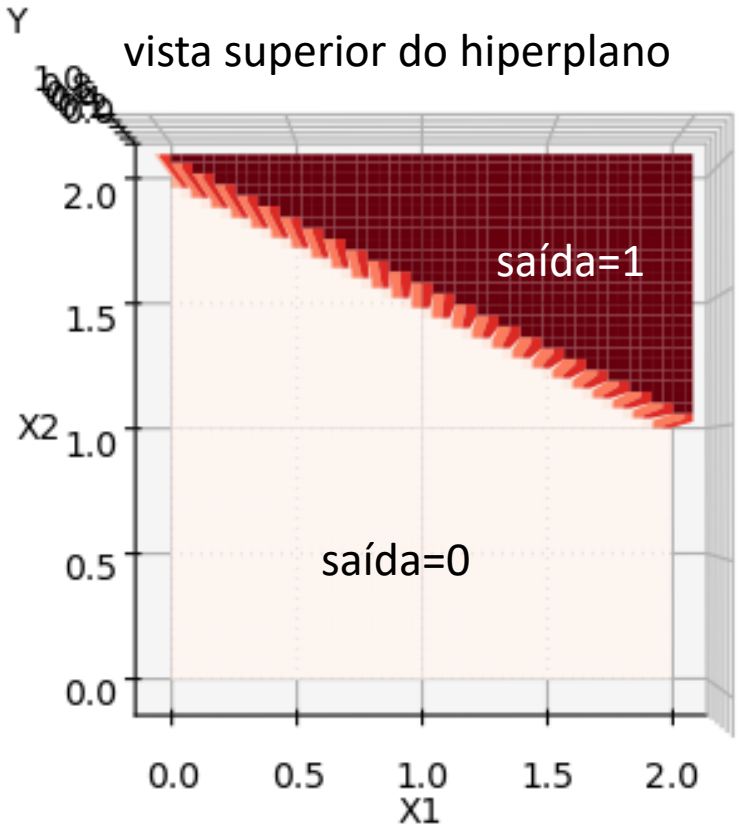
Variações nos pesos mudam a inclinação do separador ($w_1$)




$w_1 = w_2 = 1$
$\theta = 2$

1.0 w1 0.5

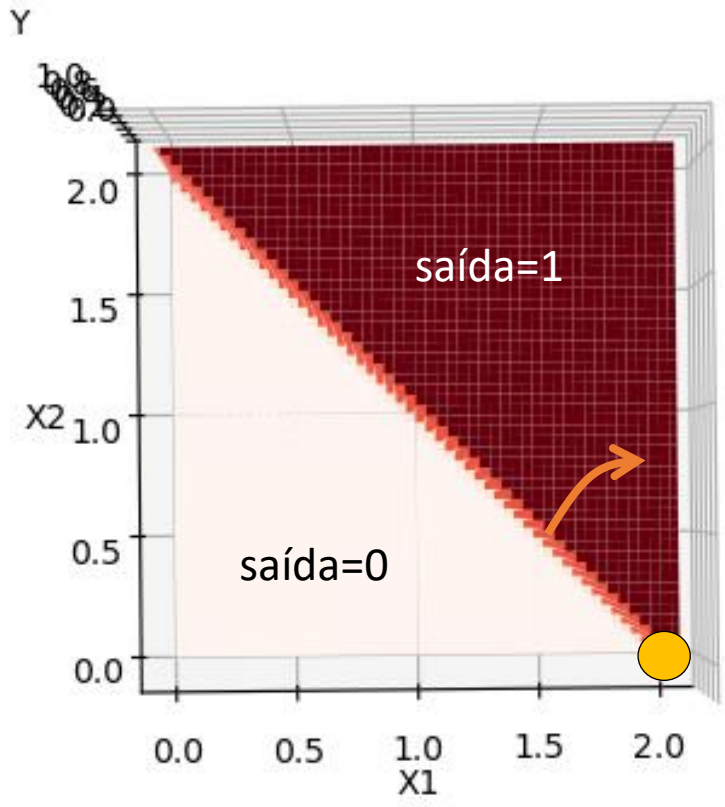
vista superior do hiperplano



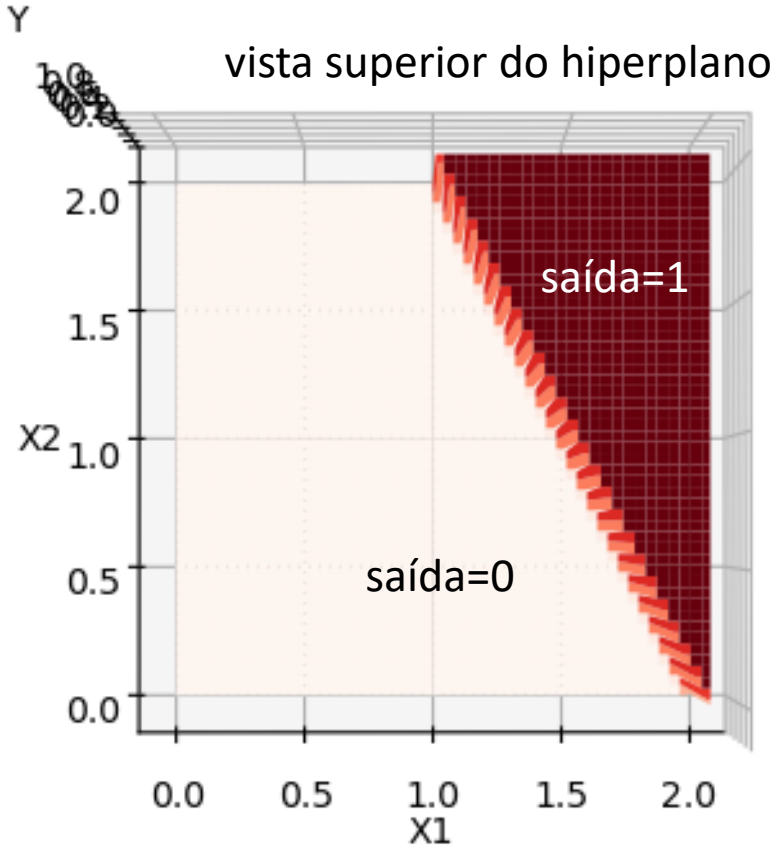

$w_1 = 0.5 \quad w_2 = 1$
$\theta = 2$

# Neurônio MCP (separação do espaço)

Variações nos pesos mudam a inclinação do separador ($w_2$)




1 w2 0.5

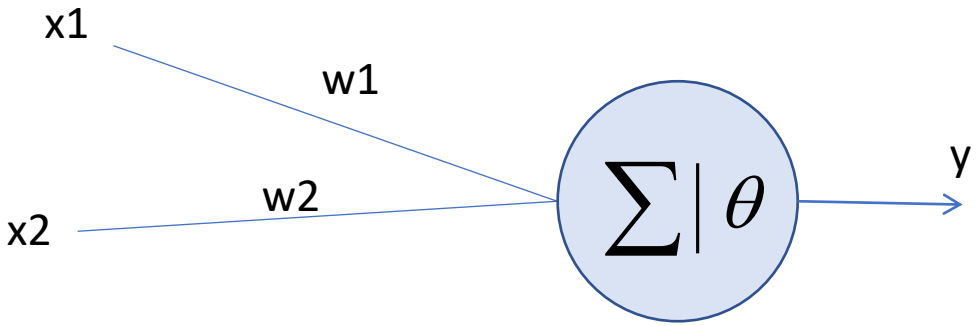



$w_1 = w_2 = 1$
$\theta = 2$

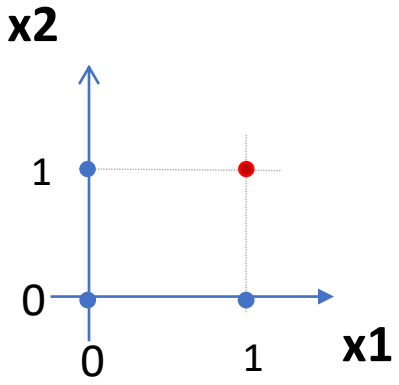
$w_1 = 1 \quad w_2 = 0.5$
$\theta = 2$

# Neurônio MCP (separação do espaço)

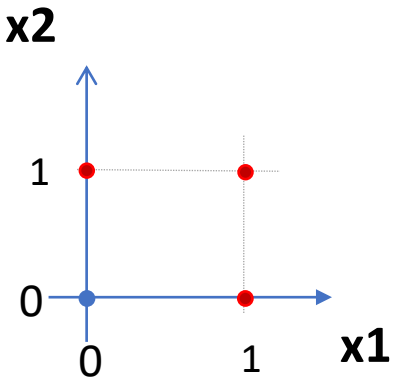
**Exercício 1**: suponha que uma RNA McP com duas entradas X1 e X2 ∈ {0, 1}. Defina os pesos w1 e w2 e o limiar θ para definir os operadores AND e OR



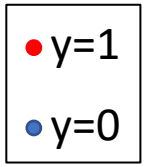

$$y = \begin{cases} 1 & \text{se} \quad w_1 x_1 + w_2 x_2 \geq \theta \\ 0 & \text{caso contrário} \end{cases}$$




**AND**




**OR**



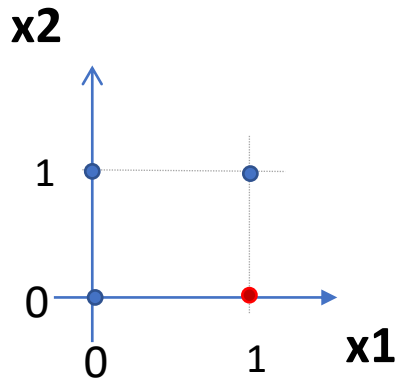
# Neurônio MCP (separação do espaço)

**Exercício 2**: suponha um neurônio MCP com uma entrada X1 ∈ {0, 1}. Defina o peso w1 e o limiar θ para definir o operador NOT

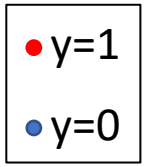
$$y = \begin{cases} 1 & \text{se } w_1 x_1 \geq \theta \\ 0 & \text{caso contrário} \end{cases}$$




**Exercício 3**: suponha um neurônio MCP com duas entradas X1 e X2 ∈ {0, 1}. Defina os pesos w1 e w2 e o limiar θ para definir X1 AND NOT X2

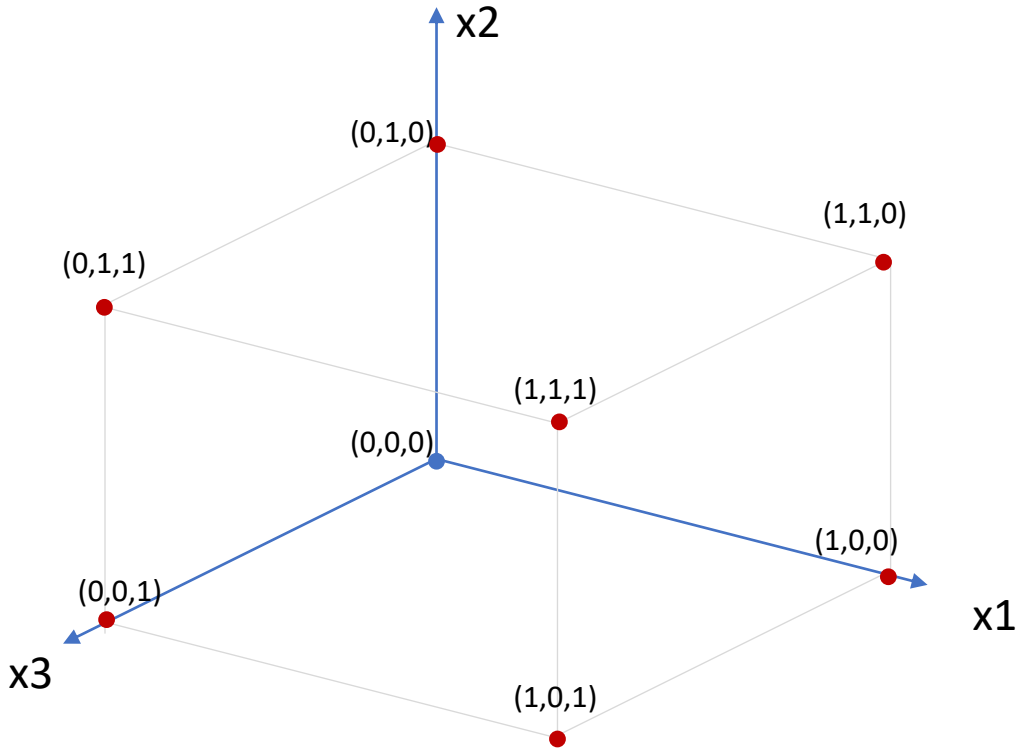



**X1 AND NOT X2**



# Neurônio MCP (separação do espaço)

**Exercício 4**: dado um neurônio McP com três entradas x1, x2 e x3 $\in \{0, 1\}$, defina os pesos w1, w2 e w3 e o limiar $\theta$ para definir x1 OR x2 OR x3




y=1

y=0

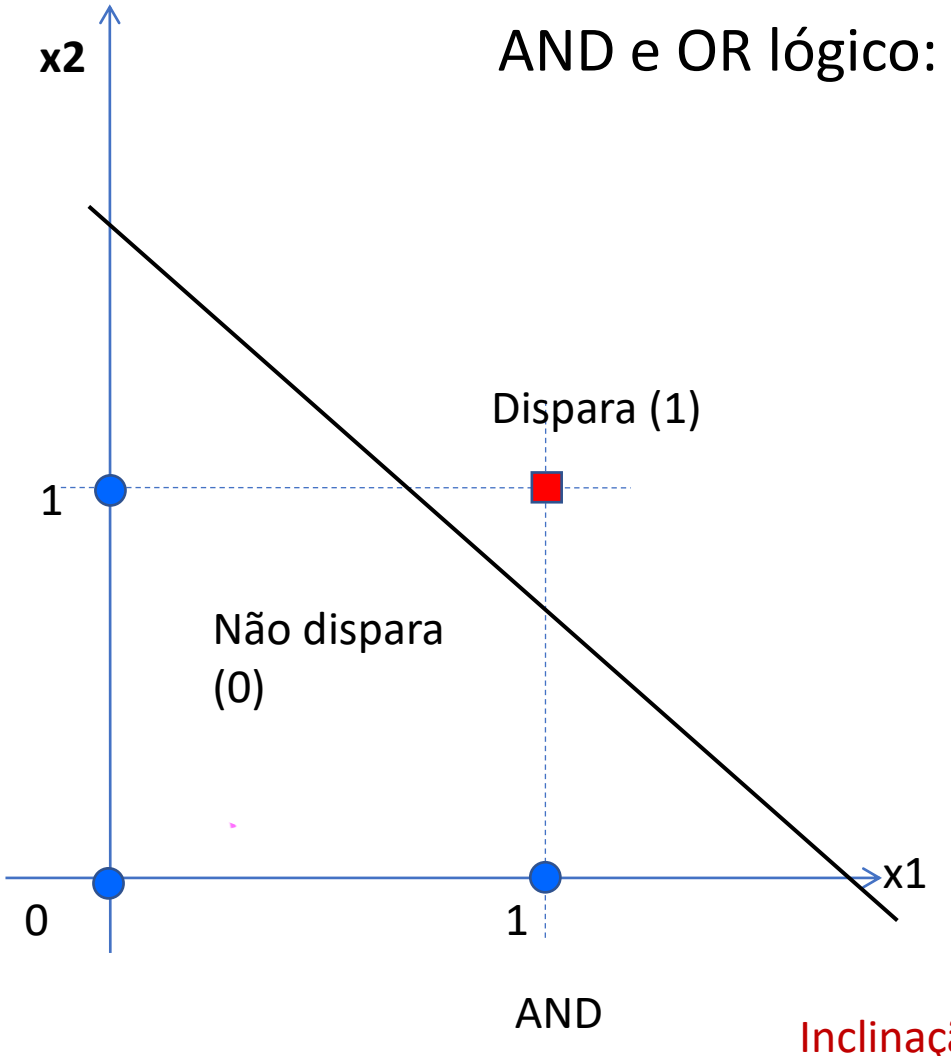
# Neurônio MCP (separação do espaço)

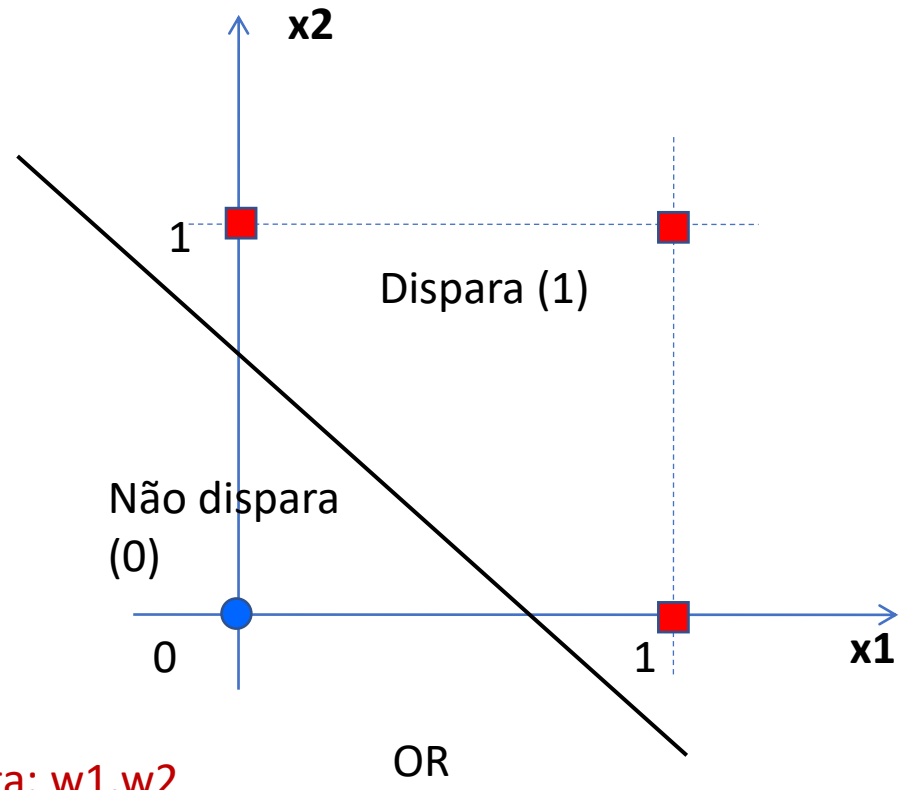
Nos exercícios, você pode observar que o neurônio MCP é capaz de representar os operadores lógicos que são separáveis linearmente.

**Separação linear: há um ponto** (1 entrada), **uma reta** (2 entradas), **plano** (3 entradas) ou **hiperplano** (4 ou +entradas) que separa o espaço da entrada em dois.

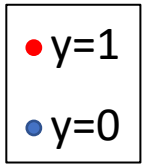
# Neurônio MCP (partição do espaço)

AND e OR lógico: linearmente separáveis



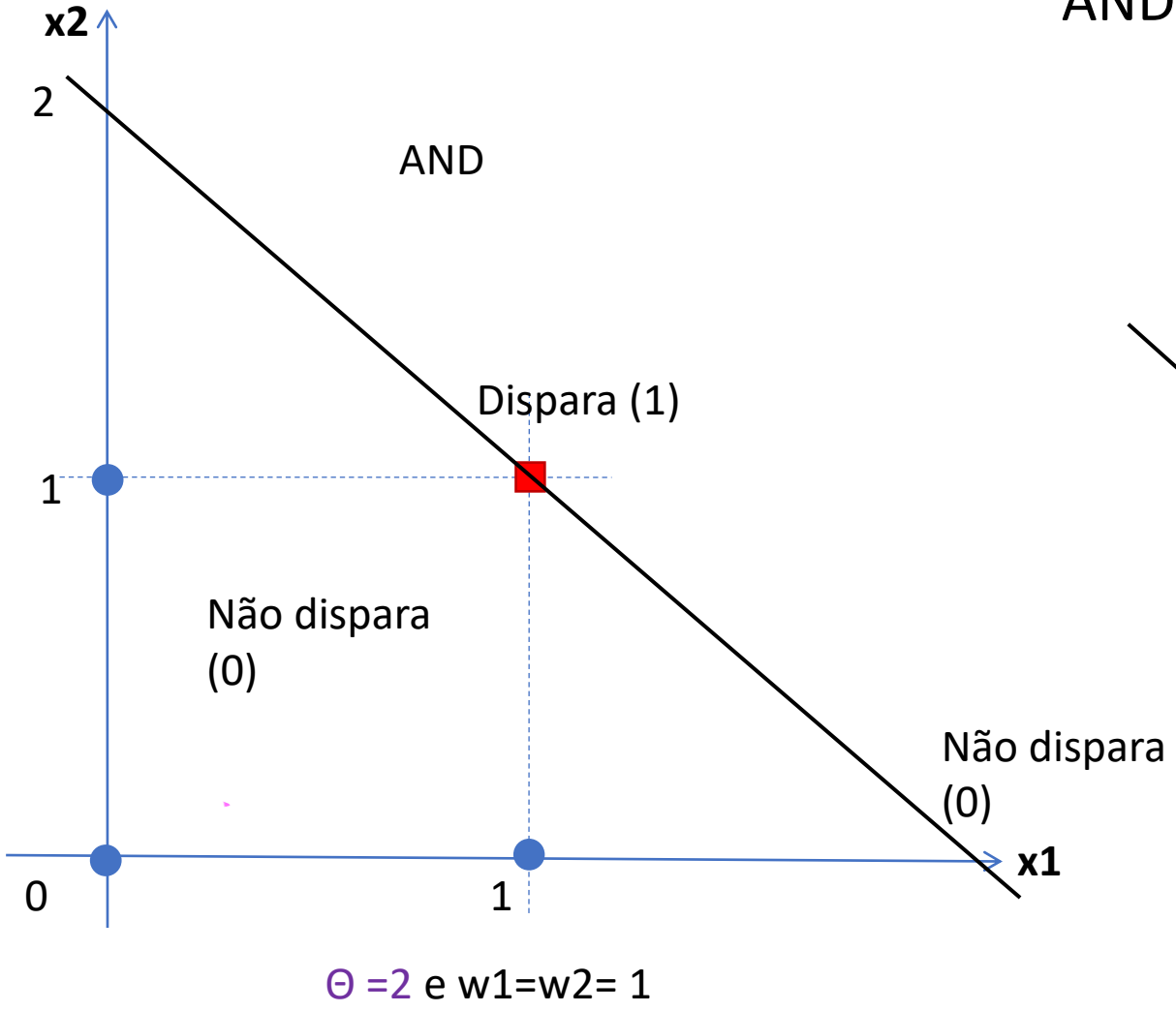




Inclinação da reta: w1,w2
Translação da reta: Θ

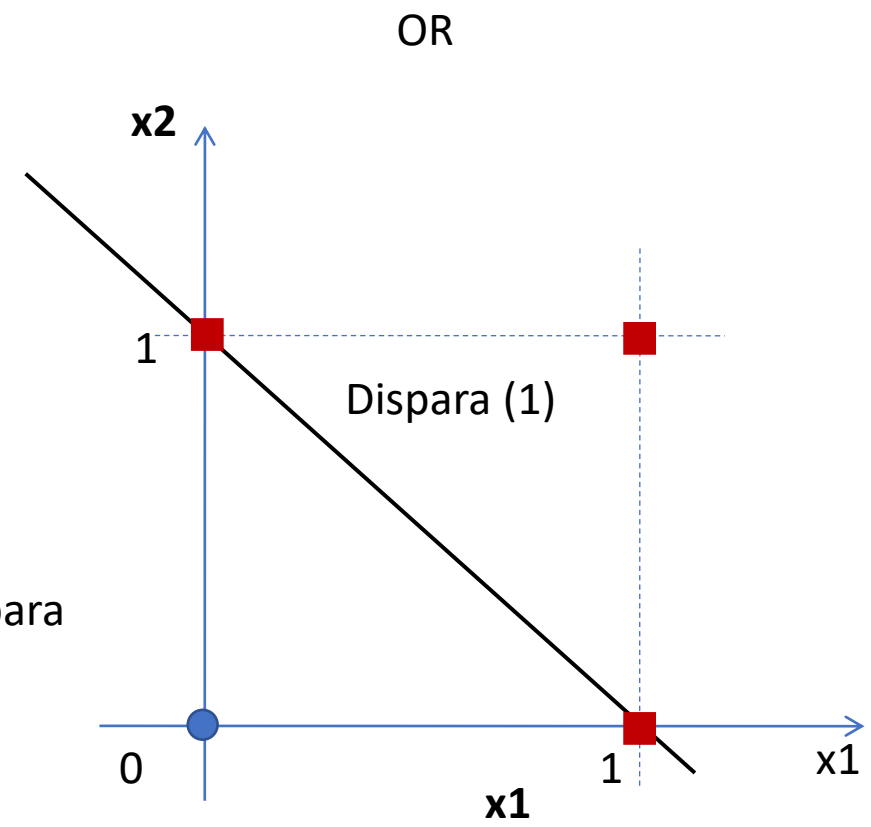


# RNAs: Neurônio MCP (partição do espaço)

AND , OR lógico: linearmente separáveis




Θ =2 e w1=w2= 1

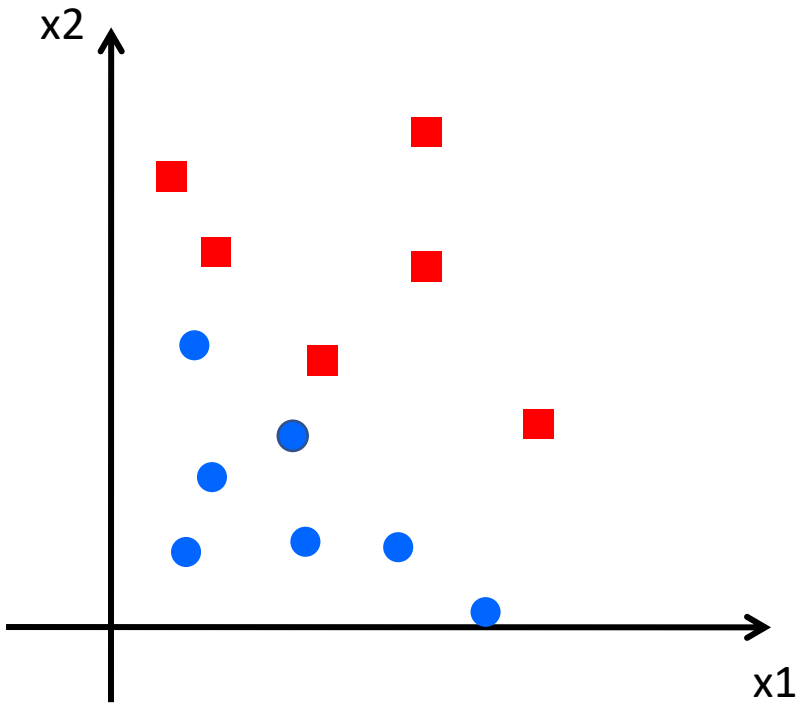



Θ =1 e w1= w2 = 1

y=1

y=0

# MCP: Problema de Classificação

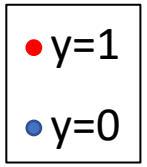



Suponha uma modelagem onde os atributos de clima anormal são x1 e x2.

Vermelhos: exemplos positivos onde se verifica anormalidade do clima
Azuis: exemplos negativos

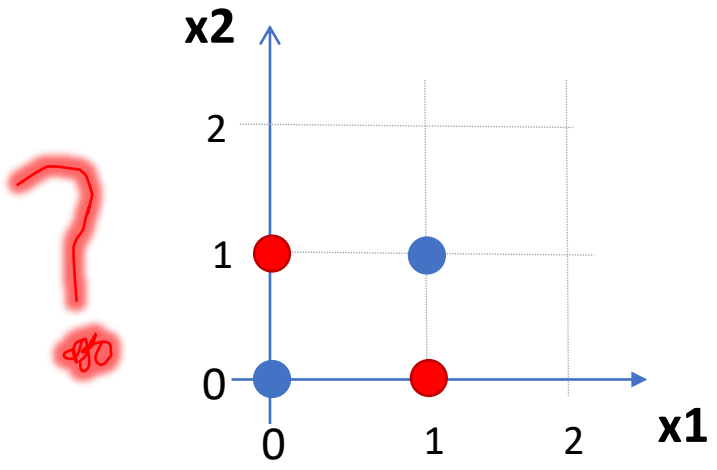
*Este problema poderia ser resolvido por um único neurônio MCP?*

*As duas variáveis (features) são necessárias para separar os positivos dos negativos?*



# Neurônio MCP (limitações)

Precisamos definir os pesos e limiares manualmente?
Onde está o aprendizado?
E as funções que não são linearmente separáveis (ex. XOR)?




XOR

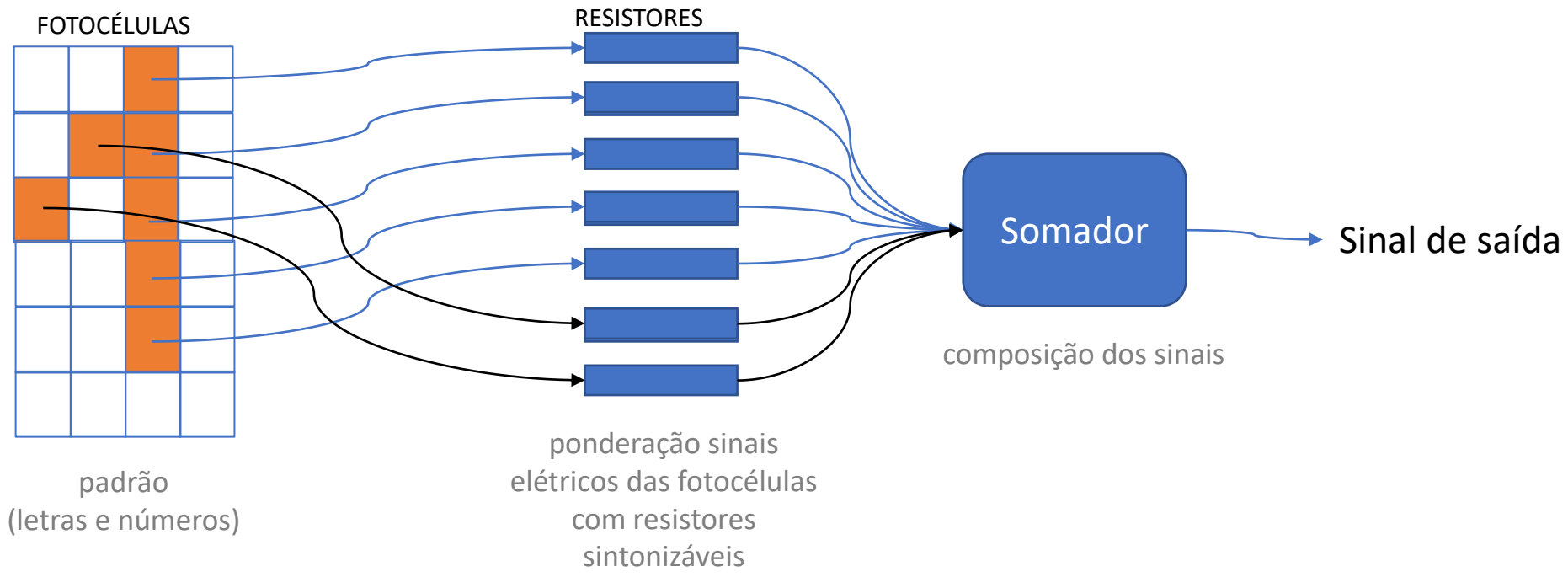
y=1
y=0

# PERCEPTRON

Modelo de neurônio artificial

# Perceptron

(Rosenblatt, 1958): construiu o neurocomputador *Mark I – Perceptron* a partir do modelo básico de Perceptron para reconhecimento de padrões geométricos inspirado na retina.




Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65, 386–408.

# Perceptron

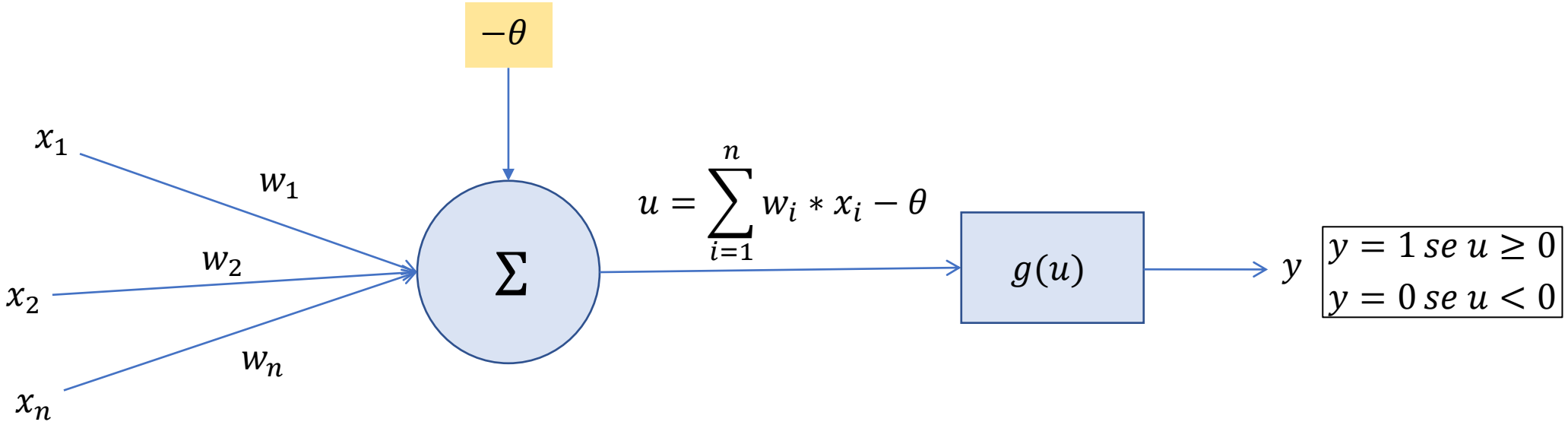
- Em vez de manipular diretamente $\theta$ (limiar), trata-o como uma das entradas ($-\theta$) tornando possível aprendê-lo

- Introduz mecanismo de aprendizagem
  (não havia em McCulloch-Pitts)

- Pertence à arquitetura *feedforward* de camada única
  (Single Layer Perceptron)

  - **Entradas**: reais ou binárias
  - **Saída**: binária (tarefa de classificação binária linearmente separáveis)
  - **Função de ativação**: degrau [0,1] ou degrau bipolar [-1,1]
  - **Treinamento**: supervisionado
  - **Regra de aprendizado**: de Hebb

# Perceptron (convenção)

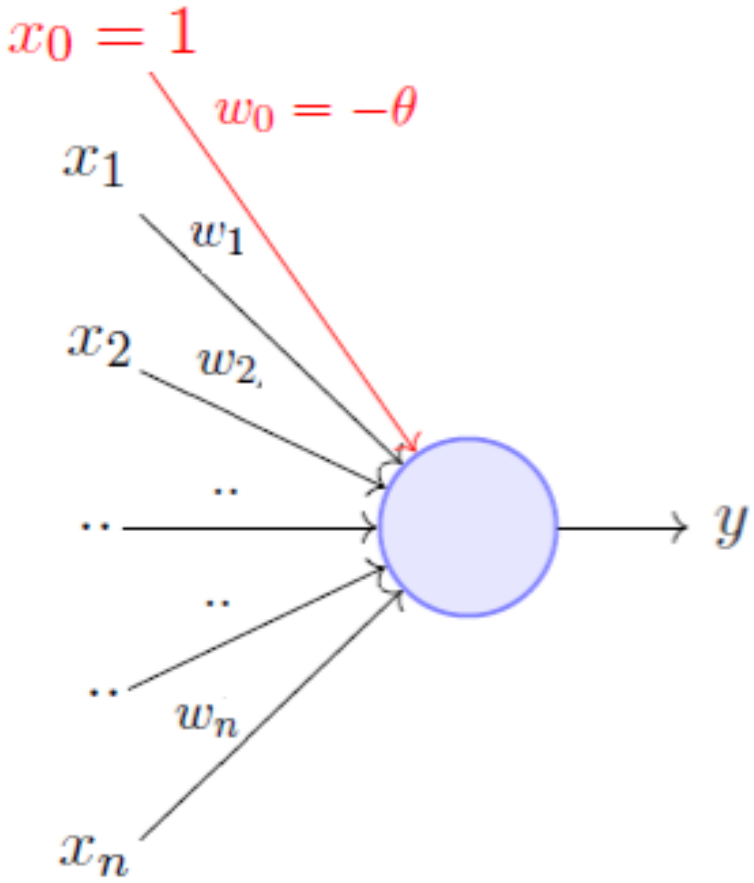



$$y = 1\, se\, \sum_{i=1}^{n} w_i * x_i \geq \theta$$
$$y = 0\, se\, \sum_{i=1}^{n} w_i * x_i < \theta$$

*MCP reescrito*

$$y = 1\, se\, \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$
$$y = 0\, se\, \sum_{i=1}^{n} w_i * x_i - \theta < 0$$

# **Perceptron**: convenção (limiar = peso de $x_0$)




introduzida nova entrada artificial $x_0$ fixada em 1

**REPRESENTAÇÃO COM VETORES**

$$\hat{x} = [x_0, x_1, x_2]$$

$$\hat{w} = [w_0, w_1, w_2]$$

$$y = \begin{cases} 1: \hat{x} \cdot \hat{w} \geq 0 \\ 0: \hat{x} \cdot \hat{w} < 0 \end{cases}$$

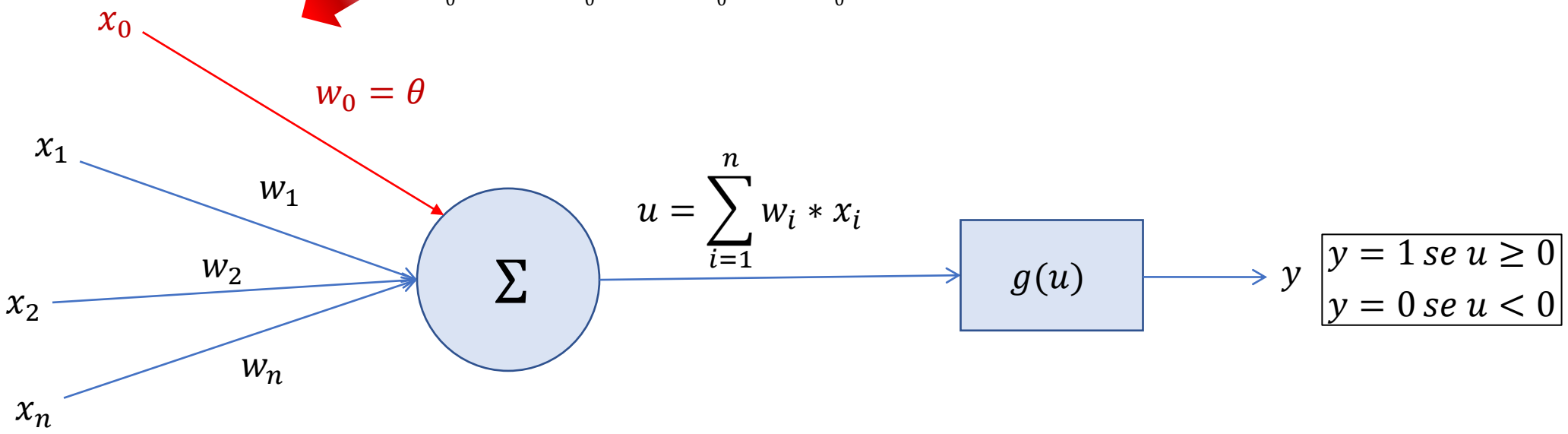
*Produto escalar do vetor de entrada * peso*

$$tal\ que\ x_0 = 1\ e\ w_0 = -\theta$$

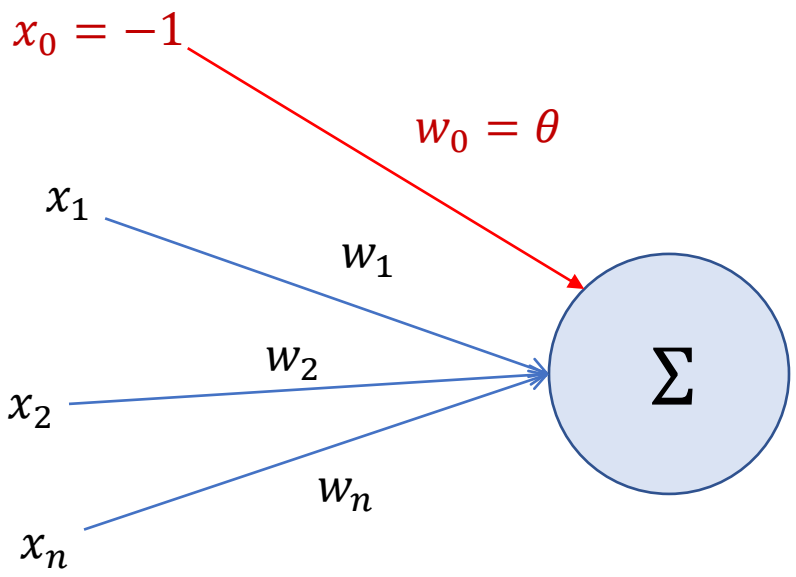
# **Perceptron**: convenção (limiar = peso de $x_0$)

Introduzimos uma entrada artificial $x_0$ sempre igual a 1 associada a um peso que representa o limiar:

$x_0 = -1\ e\ w_0 = \theta\ ou\ x_0 = 1\ e\ w_0 = -\theta$



# **Perceptron**: exemplo (*ir ao cinema*)




Supondo que as $x_i$ são booleanas:

$x_1$ = há cinema perto (sim=1, não=0)
$x_2$ = filme interessante
$x_3$ = ingresso barato

$w_0 = 0$ representa um perfil sem restrições
$w_0 = 2$ representa um perfil mais exigente quanto à proximidade, valor e interesse no filme (*duas condições devem estar presentes*)

**Exemplo perfil exigente**

$x_0 = 2, x_1 = 1, , x_2 = x_3 = 0, w_0 = -2, \ w_1 = w_2 = w_3 = 1$

$-1 * 2 + \ 1 * 1 \ + \ 1 * 0 \ + \ 1 * 0 \ \geq 0 \ ? \ falso,$

$logo \ y = 0 \ (não \ vai \ ao \ cinema)$

# **Perceptron**: exercícios

**Exercício 1:** escreva o ‘ou’ lógico para duas entradas na forma de um perceptron (com $\theta = w_0$). Quantas soluções o sistema de inequações apresenta?

**Exercício 2a**: escreva o ‘xor’ lógico para duas entradas na forma de um perceptron (com $\theta = w_0$). Quantas soluções o sistema de inequações apresenta?

**Exercício 2b**: um algoritmo de aprendizado de árvores de decisão consegue aprender um modelo de XOR? Explique o tipo de separação realizada por este tipo de algoritmo no espaço de instâncias (ID3).

# Regra de Hebb

TREINAMENTO DE PERCEPTRON

# Regra de aprendizado de Hebb

Se a saída predita é menor do que a esperada, então **excita-se** os pesos sinápticos = incrementar proporcionalmente aos sinais de entrada.

Se a saída predita é maior do que a esperada, então **inibe-se** os pesos sinápticos = decrementar proporcionalmente aos sinais de entrada

[Hebb, 1949]

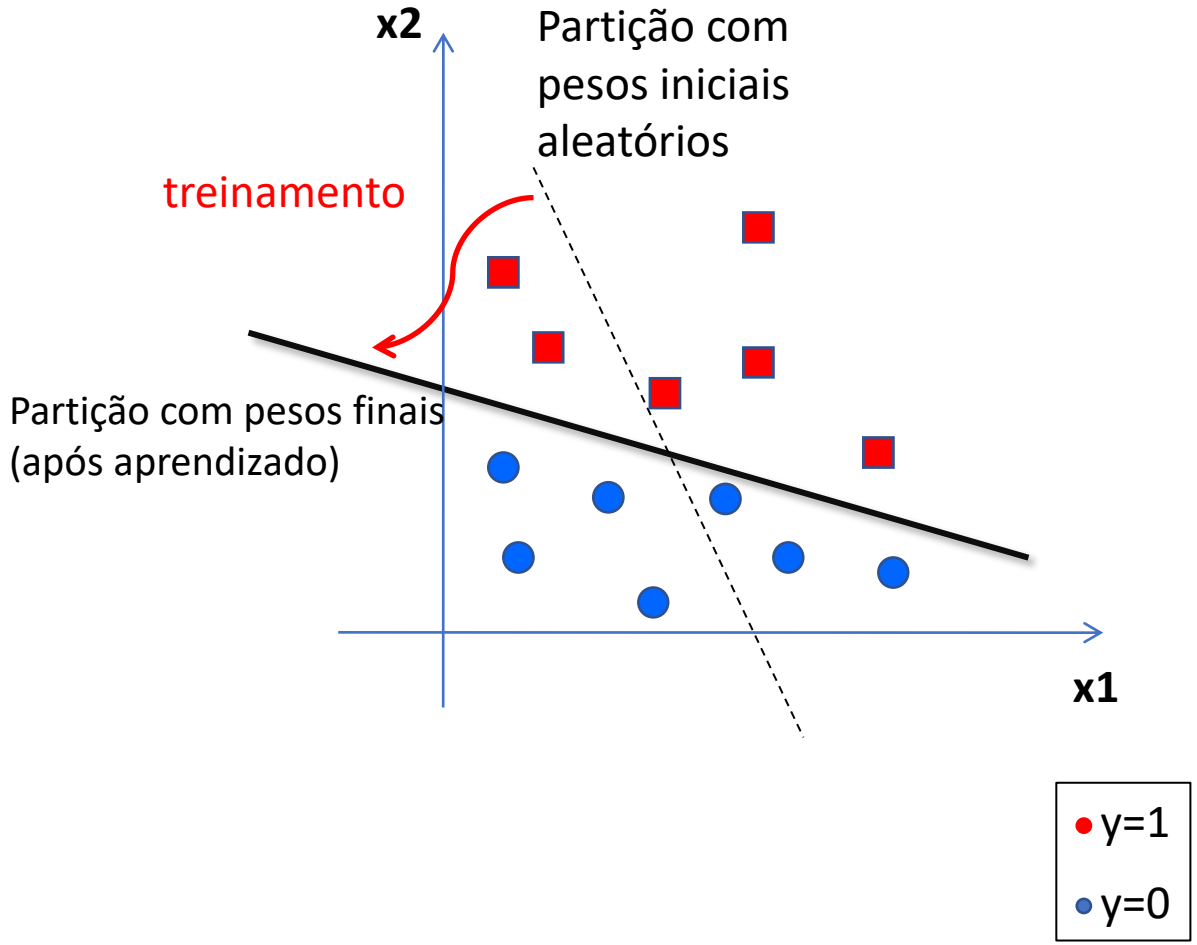
# Pesos do Neurônio (treinamento)

$-w_0 + w_1 x_1 + w_2 x_2 = 0$

$w_1 e\ w_2\ inclinam\ o\ separador\ linear$
$\theta\ translada\ o\ separador\ linear$

$$x_2 = -\left(\frac{w_1}{w2}\right).x_1 + \frac{w_0}{w_2}$$

O treinamento altera os parâmetros w1, w2 e Θ: modifica a posição da reta e, portanto, a partição do espaço de entradas.



# Regra de aprendizado de Hebb

$$\hat{w} := \hat{w} + \eta.(y' - y).\hat{x}$$

Notação algorítmica, tal que:

- $\hat{w}$ vetor de pesos do perceptrons
- $\eta$ taxa de aprendizado (*learning rate*)
- $\hat{x}$ vetor com as *features* do exemplo de treinamento
- **y'** saída esperada (*target*) para $\hat{x}$
- **y** saída produzida pelo perceptrons dado $\hat{x}$

# Regra de Hebb: exemplo de treinamento

Contexto: x1 é a nota de avaliações e x2 é o grau de participação de estudantes coletadas por um professor. O objetivo é classifica-las(los) como potenciais candidatas(os) à atividades extra-curriculares (e.g. PET, IC).

*EXEMPLOS*

| *FEATURES* | $\widehat{x_1}$ | $\widehat{x_2}$ | $\widehat{x_3}$ | $\widehat{x_4}$ |
|---|---|---|---|---|
| **x0** | -1.0 | -1.0 | -1.0 | -1.0 |
| **x1** | 0.6 | 0.8 | 0.3 | 0.9 |
| **x2** | 0.7 | 0.4 | 0.8 | 1.0 |
| **y'** | 1 | 0 | 0 | 1 |

*PESOS* $\widehat{w}$

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.0 | 0.8 | 0.1 |

Aprendizado dos pesos por época

**Entradas (features)**
x0 = entrada do bias
x1 = nota [0, 1]
x2 = participação [0, 1]

Observar que o peso w1 (notas) está bem maior que w2 (participação).
O ideal é um equilíbrio entre quantidade de exercícios e ingestão de calorias.

# Regra de Hebb: exemplo de treinamento

ÉPOCA 1: Apresentação do exemplo $\widehat{x_1}$

*EXEMPLOS*

| *FEATURES* | $\widehat{x_1}$ | $\widehat{x_2}$ | $\widehat{x_3}$ | $\widehat{x_4}$ |
|---|---|---|---|---|
| **x0** | -1.0 | -1.0 | -1.0 | -1.0 |
| **x1** | 0.6 | 0.8 | 0.3 | 0.9 |
| **x2** | 0.7 | 0.4 | 0.8 | 1.0 |
| **y'** | 1 | 0 | 0 | 1 |

*PESOS* $\widehat{w}$

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.0 | 0.8 | 0.1 |

***disparo***: $u = \widehat{x_1}.\widehat{w}$

$u = (0.0 + 0.48 + 0.07) = 0.55$

$0.55 \geq 0, logo\ y = 1$

***erro***: $y' = y$ então $e = 1 - 1 = 0$

***ajuste***: $\eta.e.\hat{x}$ $com\ \eta = 0.05$

[0.0 0.0 0.0]

*PESOS* $\widehat{w}$ *atualizados*

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.0 | 0.8 | 0.1 |

# Regra de Hebb: exemplo de treinamento

ÉPOCA 1: Apresentação do exemplo $\widehat{x_2}$

*EXEMPLOS*

| *FEATURES* | $\widehat{x_1}$ | $\widehat{x_2}$ | $\widehat{x_3}$ | $\widehat{x_4}$ |
|---|---|---|---|---|
| **x0** | -1.0 | -1.0 | -1.0 | -1.0 |
| **x1** | 0.6 | 0.8 | 0.3 | 0.9 |
| **x2** | 0.7 | 0.4 | 0.8 | 1.0 |
| **y'** | 1 | 0 | 0 | 1 |

*PESOS* $\widehat{w}$

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.0 | 0.8 | 0.1 |

***disparo***: $u = \widehat{x_1}.\widehat{w}$

$u = (0.0 + 0.64 + 0.04) = 0.68$

$0.68 \geq 0, logo\ y = 1$

***erro***: $y' \neq y$ então $e = 0 - 1 = -1$

***ajuste***: $\eta.e.\hat{x}$ $com\ \eta = 0.05$

[0.05 -0.04 -0.02]

*PESOS* $\widehat{w}$ *atualizados*

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.05 | 0.76 | 0.08 |

# Regra de Hebb: exemplo de treinamento

ÉPOCA 1: Apresentação do exemplo $\widehat{x_3}$

*EXEMPLOS*

| *FEATURES* | $\widehat{x_1}$ | $\widehat{x_2}$ | $\widehat{x_3}$ | $\widehat{x_4}$ |
|---|---|---|---|---|
| **x0** | -1.0 | -1.0 | -1.0 | -1.0 |
| **x1** | 0.6 | 0.8 | 0.3 | 0.9 |
| **x2** | 0.7 | 0.4 | 0.8 | 1.0 |
| **y'** | 1 | 0 | 0 | 1 |

*PESOS* $\widehat{w}$

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.05 | 0.76 | 0.08 |

$\boldsymbol{disparo}$: $u = \widehat{x_1}.\widehat{w}$

$u = (-0.05 + 0.23 + 0.064) = 0.242$

$0.292 \geq 0, logo\ y = 1$

$\boldsymbol{erro}$: $y' \neq y$ então $e = 0 - 1 = -1$

$\boldsymbol{ajuste}$: $\eta.e.\hat{x}$ $com\ \eta = 0.05$

[ 0.05 -0.015 -0.04]

*PESOS* $\widehat{w}$ *atualizados*

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.1 | 0.745 | 0.04 |

# Regra de Hebb: exemplo de treinamento

ÉPOCA 1: Apresentação do exemplo $\widehat{x_4}$

*Quando todos os exemplos de treino são apresentados = fim da época*

*EXEMPLOS*

| *FEATURES* | $\widehat{x_1}$ | $\widehat{x_2}$ | $\widehat{x_3}$ | $\widehat{x_4}$ |
|---|---|---|---|---|
| **x0** | -1.0 | -1.0 | -1.0 | -1.0 |
| **x1** | 0.6 | 0.8 | 0.3 | 0.9 |
| **x2** | 0.7 | 0.4 | 0.8 | 1.0 |
| **y'** | 1 | 0 | 0 | 1 |

*PESOS* $\widehat{w}$

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.1 | 0.745 | 0.04 |

***disparo***: $u = \widehat{x_1}.\widehat{w}$

$u = (-0.1 + 0.67 + 0.04) = 0.61$

$0.61 \geq 0, logo\ y = 1$

***erro***: $y' = y$ então $e = 1 - 1 = 0$

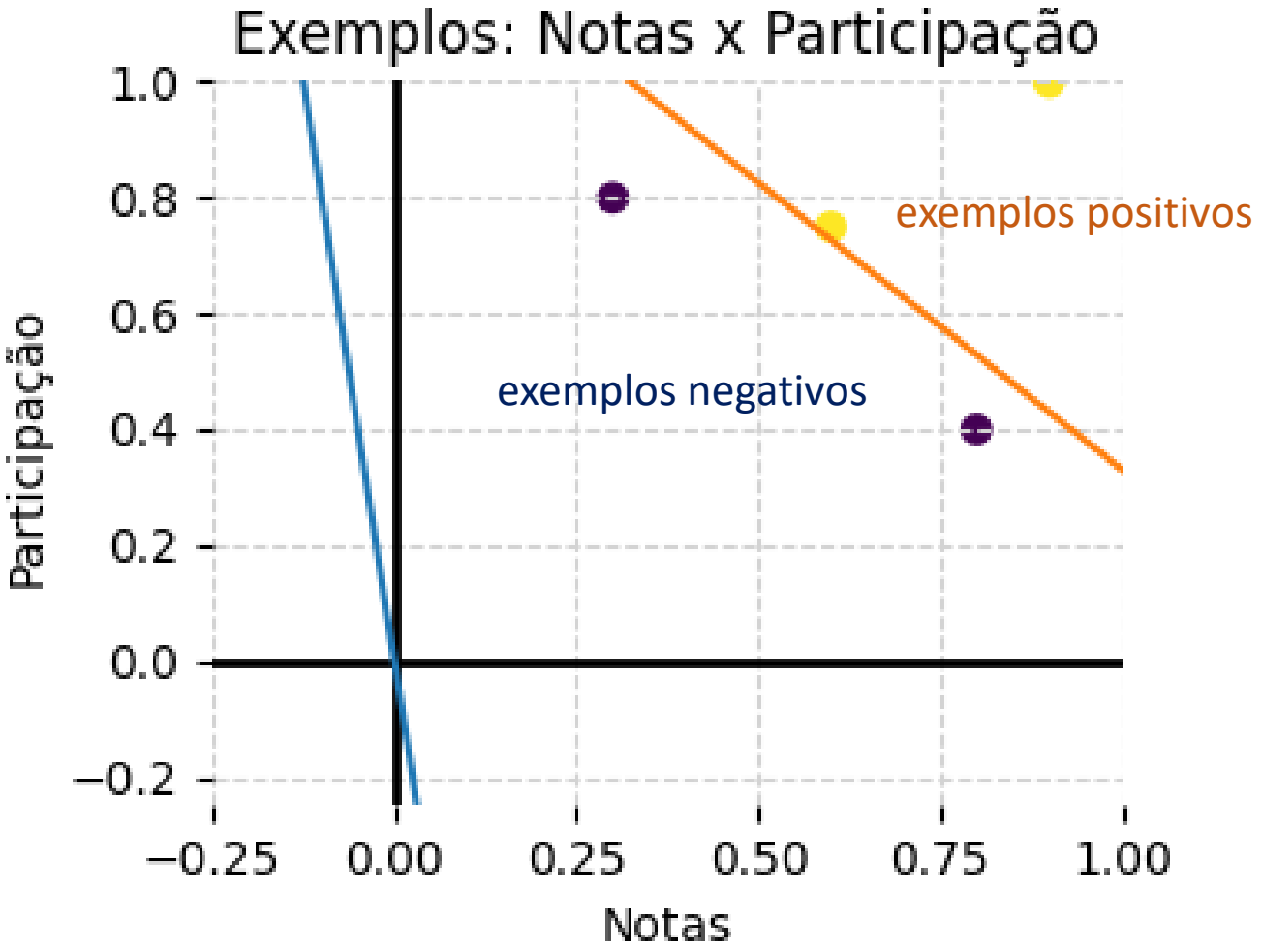
***ajuste***: $\eta.e.\hat{x}$ $com\ \eta = 0.05$

[ 0.00 0.00 0.00]

*PESOS* $\widehat{w}$ *atualizados*

| **w0** | **w1** | **w2** |
|---|---|---|
| 0.1 | 0.745 | 0.04 |

# Regra de Hebb: modelo aprendido




*Linha azul*
separador inicial

| w0 | w1 | w2 |
|---|---|---|
| 0.0 | 0.8 | 0.1 |

*Linha laranja*
separador com os pesos finais após **30 épocas**

| w0 | w1 | w2 |
|---|---|---|
| 0.4 | 0.3 | 0.3 |

Observe que w1 se igualou a w2, as 2 entradas são importantes

# Treinamento de Perceptron com regra de Hebb

```
η: taxa de aprendizado (constante, valor  [0,1])
MAX_EPOCAS: número máximo de épocas (constante)
x: instâncias de treinamento (entrada) tal que
   x[i] = [-1 x1 x2 ... xn]  i-ésima instância de treino com features de x1 a xn
y': saída esperada para uma instância (entrada, indexada)
   y'[i] = saída esperada para a i-ésima instância
w: vetor de pesos (único) = [θ  w1  w2 …  wn]

época := 0
inicializar w com valores aleatórios entre [-1,1]
faça {
  para cada instância i de treinamento {
    erro := false
    u := wT.x[i]        # produto escalar dos vetores
    y := g(u)           # predição da saída (calculada)
    se y ≠ y'[i] {    # saída calculada difere da esperada
       erro=true
       w := w + η.(y'[i] – y).x[i]
    }
  }
   épocas++    #passou por todas as instâncias
} enquanto épocas < MAX_EPOCAS or not erro
```

# Perceptron: pontos positivos e negativos

- Só realiza separação linear, caso contrário treinamento não converge
- Não atualiza mais os pesos assim que todas as instâncias são classificadas corretamente: *pode perder capacidade de generalizar*
- *Pequenas mudanças nos pesos, podem fazer a saída 'virar' de 1 para zero ou vice-versa*

*Uma utilidade do perceptron é que pode ser utilizado para testar se um dataset é linearmente separável!!!*

*Se as instâncias positivas e negativas são linearmente separáveis, o algoritmo comprovadamente converge após um número finito de iterações.*

# Adaline com Regra Delta

REDE ADALINE E TREINAMENTO DE PERCEPTRON

# Adaline

Adaline [Widrow, Hoff, 1960]

Treinamento feito com a **regra Delta** (Least Mean Square ou Descida de Gradiente)
**(Precursor do algoritmo de regra Delta generalizado utilizado em redes multicamadas)**

Aplicação prática (original): comutações de circuitos telefônicos

REGRA DELTA

É um algoritmo de aprendizado supervisionado de **classificadores binários**. Busca minimizar o número de instâncias mal classificadas modificando o peso das entradas e o limiar de ativação do neurônio.

# Adaline

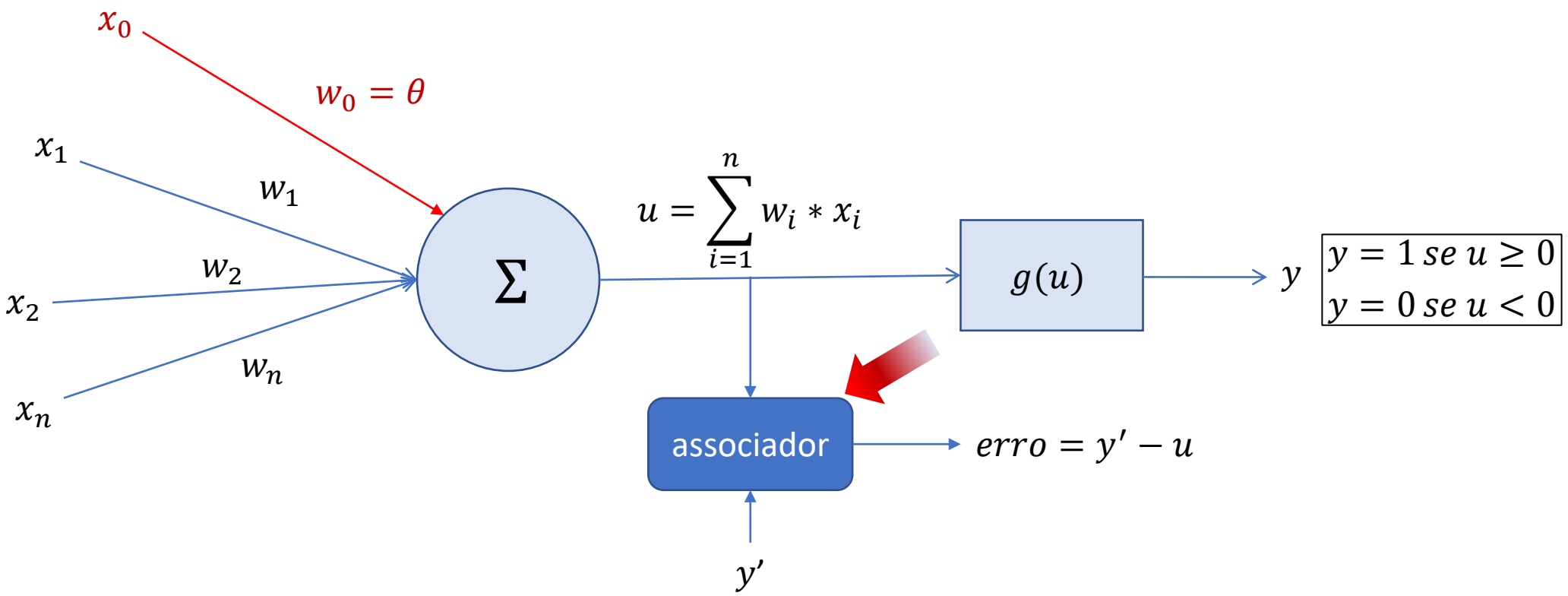
- Em vez de manipular diretamente $\theta$ (limiar), trata-o como uma das entradas ($-\theta$) tornando possível aprendê-lo (*idem Perceptron)*

- **Passa a utilizar a regra Delta no <u>mecanismo de aprendizagem</u>**
  (Perceptron utiliza regra de Hebbs)
  Inclui um bloco associador para auxiliar node treinamento: o erro será dado por y' - u

- Pertence à arquitetura *feedforward* de camada única
  *(idem à Single Layer Perceptron)*

  - **Entradas**: reais ou binárias
  - **Saída**: binária (tarefa de classificação binária para classes linearmente separáveis)
  - **Função de ativação**: degrau [0,1] ou degrau bipolar [-1,1]
  - **Treinamento**: supervisionado
  - **Regra de aprendizado**: **Delta**

# Rede Adaline



# Treinamento com Regra Delta

Baseado na descida de gradiente (ou *Least Mean Square*)

A ideia é utilizar a diferença $(y' - u)$ para ajustar o vetor de pesos $w$ para minimizar o **erro quadrático** considerando todo o conjunto de instâncias de treinamento.

$$E(w) = \frac{1}{2}\sum_{k=1}^{p}(y'_k - u)^2$$

p é o total de instâncias de treinamento

# Treinamento com regra Delta

1) Calcula-se o vetor gradiente do erro:

$$\nabla E(w) = \frac{\partial E(w)}{\partial w}$$

2) Ajusta-se os pesos na direção oposta ao do gradiente porque queremos minimizar o erro :

$$\Delta w = -\nabla E(w)$$

3) Considere que o resultado de 1 é

$$\nabla E(w) = -\frac{1}{2}\sum_{k=1}^{\rho} (y' - u).x_k$$

4) Como a atualização dos pesos se faz por

$$w := w + \eta.\Delta \mathrm{w}$$

5) Obtemos a fórmula de atualização dos pesos a partir de 2, 3 e 4 para uma batelada $\rho$

$$w := w + \eta.\frac{1}{2}\sum_{k=1}^{\rho} (y' - u).x_k$$

6) ou a cada instância de $\rho$

$$w := w + \eta.(\mathrm{y}' - \mathrm{u}).\mathrm{x_k}$$

# Treinamento de Perceptron com regra DELTA

```
η: taxa de aprendizado (constante, valor  [0,1])
MAX_EPOCAS: número máximo de épocas (constante)
x: instâncias de treinamento (entrada) tal que
   x[i] = [-1 x1 x2 ... xn]  i-ésima instância de treino com features de x1 a xn
y': saída esperada para uma instância (entrada, indexada)
   y'[i] = saída esperada para a i-ésima instância
w: vetor de pesos (único) = [θ  w1  w2 …  wn]

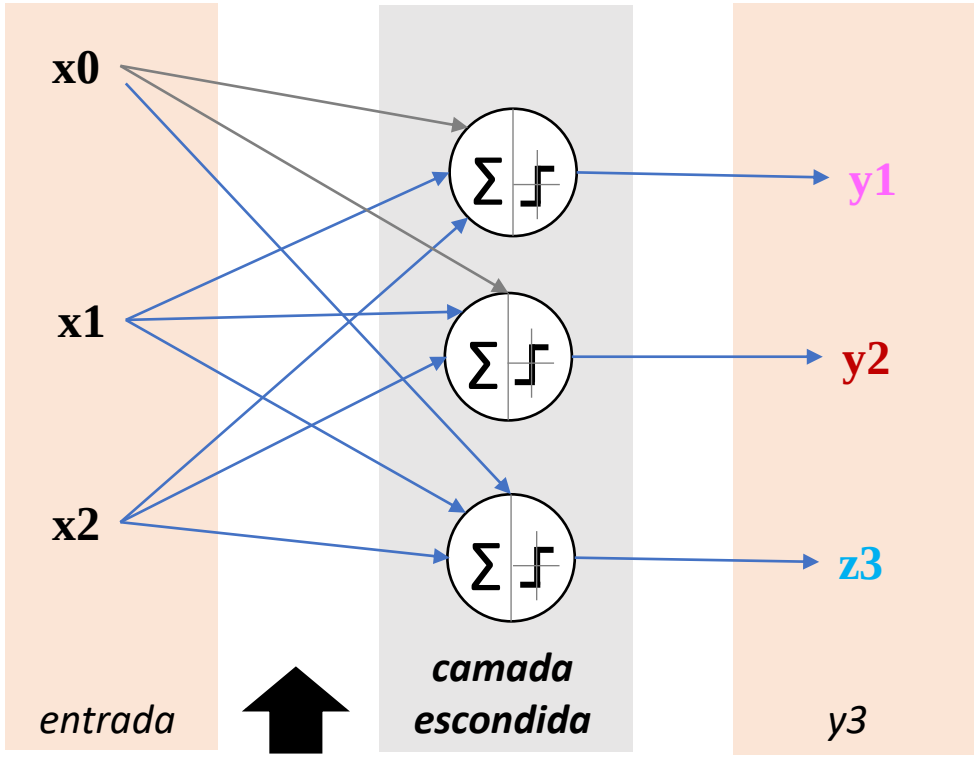
época := 0
inicializar w com valores aleatórios entre [-1,1]
faça {
  erro_ant = E(w)
  para cada instância i de treinamento {
    u := wT.x[i]          # produto escalar dos vetores
    y := g(u)             # predição da saída (calculada)
    w := w + η.(y'[i] – u).x[i]
  }
  épocas++    #passou por todas as instâncias
  erro_atual = E(w)
} enquanto épocas < MAX_EPOCAS or |erro_ant – erro_atu| ≤ ϵ
```

# REDES DE NEURÔNIOS

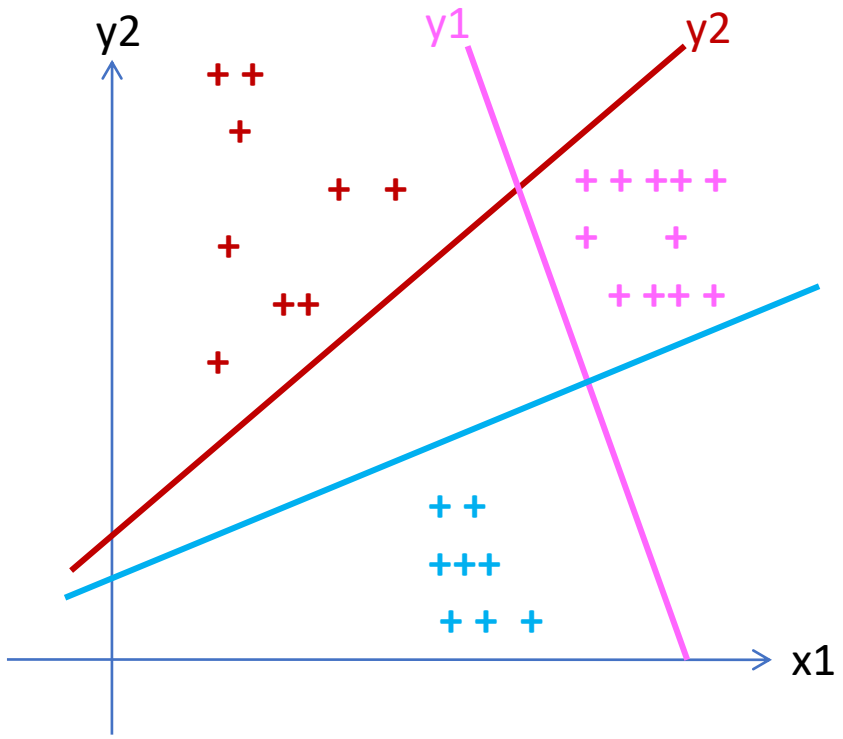
REDES NEURAIS ARTIFICIAIS

# Camada única de perceptron – Single Layer Perceptron (SLP)

São redes de uma ÚNICA camada de neurônios (*single-layer perceptron* – SLP) sem conexões retroativas (*feed forward*). Cada neurônio faz uma separação independente do espaço de entradas




só há uma camada de pesos



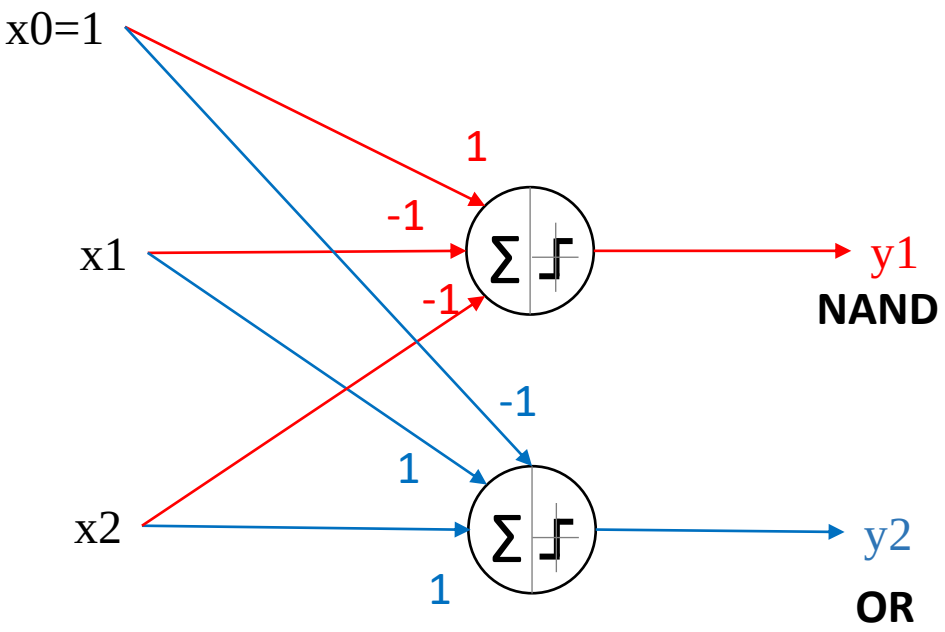

y1, y2 e y3 fazem separações independentes (ex. Madaline)

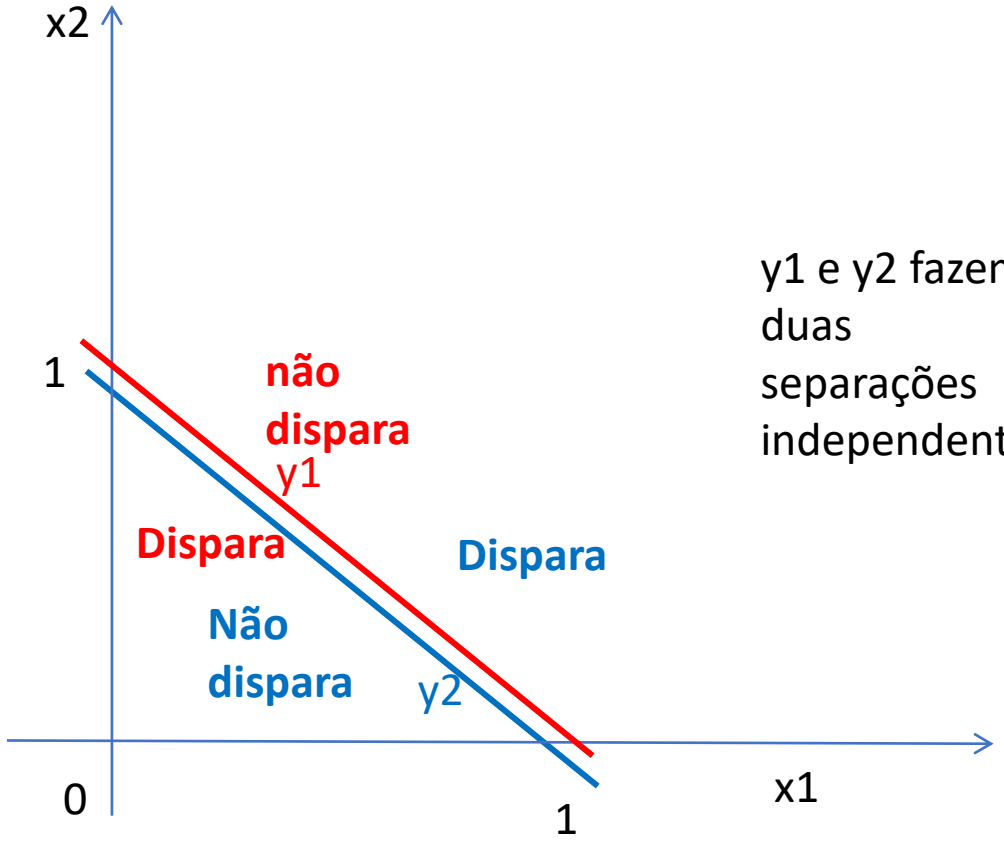
Cada neurônio age independente do outro!!!

# 1-CAMADA DE PERCEPTRONS

(*single-layer perceptron*)

Um perceptron por si só  não resolve o problema do XOR,
mas e se colocarmos dois perceptrons?




y1 e y2 fazem duas separações independentes

# Single-layer perceptron: limitações

São redes de uma só camada de pesos (*single-layer perceptron* – SLP)

Cada neurônio faz uma partição independente no espaço de entradas

Esta limitação foi apontada por Minsky e Papert (1969).

**Como faço para agrupar as saídas de cada neurônio?**



multi-layer perceptron

Minsky, M.L., Papert, S. A. (1969); Perceptrons: an introduction to computational geometry. The MIT Press, Cambridge, MA, USA.